

ЭВМ

VG6

25 декабря 2025 г.

1 До этого мне было лень

Вопросики (23 октября 2025)

Какой принцип Неймана позволяет привесить скорость света?

Вычислительный процесс следует организовывать параллельным образом.

Почему ЭВМ в двоичной системе счисления, а не в десятичной?

5 принцип Неймана: разработка устройств для других операций нецелесообразно (кроме сумматора). В теории он прав, а на практике нет. К примеру аппаратные умножители.

Главный принцип Неймана: и программа и данные лежат в одном и том же месте в одном и том же виде. Не нужно для замены машины, можно заменить только программу.

Схема - графическое обозначение элементов и связи между ними.

2 Структура классической ЭВМ.

Состоит из 3х блоков.



2.1 Описание блоков.

1. Арифметико-Логическое устройство

- Выполняет преобразование информации представленной в двоичном виде. Например: числа при выполнении арифметической операции.
- **Операнд** - мы будем называть участника арифметической или логической операции.
- АЛУ преобразует операнды и получает результат и признаки результат (дополнительные сведения о том, какой результат получился Правильный/Неправильный).
- Какой операцией заняться определяют управляющие сигналы.
- **Операция** - арифметическое или логическое действие выполняемое в АЛУ над операндами. То, что может делать АЛУ называется операцией.

2. Запоминающее устройство

- Предназначение: хранить двоичные числа.
Для этого нужны 3 операции:
 - (a) Запись
 - (b) Чтение
 - (c) Выборка (адресация не совсем одно и тоже)
- Запоминающее устройство классической ЭВМ является адресным устройством. Каждое число хранится в ячейке имеющей свой уникальный адрес, который тоже является числом. В простейшем случае - порядковый номер ячейки памяти.
- Запоминающее устройство представляет собой одномерный массив, где в качестве индекса выступает адрес (номер ячейки).
- 4 действия (тоже + хранение) определяется управляющими сигналами, среди которых сигналы: операции записи, чтения и многое другое.

3. Устройство Управления

- Занимается управлением вычислительным процессом. То есть его выходом является порождение управляющих сигналов (сигналов управления), которые потребляются другими ящиками и самим собой.
- Помимо внешних блоков он управляет и самим собой.
- Цели и задачи управления: реализация выполнения команды. Для этого формируемые им сигналы несут 2 сущности: что делать, когда это надо сделать.
- Эту команду откуда-то надо взять. А они лежат в запоминающем устройстве, там же, где и данные.
- На выходе сигналы, на входе ... из памяти
- нужно формировать адрес ячейки откуда взять команду, получить её и выполнить.

Связи между этими блоками изображены на рисунке. По признакам результата УУ влияет на дальнейшее прохождение вычислительного процесса (изменять последовательность выполнения команд).

Например, произошло переполнение, значит нужно сформировать признак переполнения и передать УУ.

УУ получает команды из памяти, однако и операнды и команды лежат в разных ячейках запоминающего устройства (адресам) поэтому УУ должен формировать адреса как команд, так и операнд.

2.2 Цикл выполнения команды.

Структура организована циклически. И этот бег по кругу является бесконечным повторением.

2.3 Память. Запоминающее устройство.

Строится по иерархическому принципу. **Регистр** - запоминающее устройство ёмкостью в 1 число. Есть 2 типа:

1. Специализированные регистры, функции которых предопределены конструкцией ЭВМ и являются неизменными и регистры
2. Общего назначения, функционал которых может предопределяться. Доступны для программистов.

2.3.1 Виды памяти

Сверхоперативные ЗУ - обычно безадресного типа. К таким устройствам относятся буферные запоминающие устройства, стек.

Буфер - запоминающее устройство между ЗУ с разной скоростью работы для сглаживания по времени. Обычно организуется очередью (FIFO).

Стек - первым вылетает последний заряженный патрон. (LIFO). История перехода к подпрограммам. Используется в системе прерываний.

Постоянное запоминающее устройство (ROM) - адресное запоминающее устройство без функции записи. **EPROM** - оставляет возможность сменить содержимое путём перезаписи, что требует специальное устройство (программатор).

КЭШ L1, L2, L3, L4 - безадресное ассоциативное запоминающее устройство. Небольшая ёмкость, нократно ускоряет скорость работы вычислительного устройства. Благодаря ассоциативному доступу (тэгами) аккумулируют в себе те команды или данные, которые используются наиболее интенсивно. Тем самым создавая копию ячеек Оперативной памяти кратно уменьшают доступ к этим данным.

Основная оперативная память ООП, ОЗУ, RAM, память с произвольной выборкой, память ЭВМ - (синенькое на схеме) память, в которой хранятся те самые команды и данные по Нейману.

Специализированные блоки памяти - обмен между вычислительным ядром внешним миром (многопортовая память, ассоциативные ЗУ (используется в КЭШе и т.д. при поиске не по адресу, а по признаку), видеопамять)

Внешние запоминающие устройства - то, что подключается к ... через интерфейс. (... , облачные зранилища, Data-центры, ...)

2.4 ОЗУ. Оперативное запоминающее устройство.

Совокупность ячеек, пре данный вид памяти для работы в качестве основной оперативной памяти ЭВМ дляжно обладать свойством произвольной выборки.

Памятью с произвольной выборкой мы будем называть адресное ЗУ, время выборки которого не зависит от адреса ячейки и последовательности обращений к ячейкам этого устройства.

Технические характеристики ЗУ:

- 1 бит - 1 двоичный разряд
- 1 байт - 8 бит. Попытка представить символы алфавита при помощи таблички кодирования (ASCII).
- К(Кило) - $2^{10} = 1024$
- М(Мега) - $2^{20} = ...$
- Г(Гига) - $2^{30} = ...$

Память - информационная ёмкость 1 адресуемой ячейки того, что имеет адрес.

Организация ЗУ - произведение числа ячеек на их разрядность, например: $4\text{Г} \times 8$

Характеристики запоминающего устройства - временные.

Быстродействие (производительность) ЗУ - оценивают временами считывания и записи, длительностью циклов считывания/записи

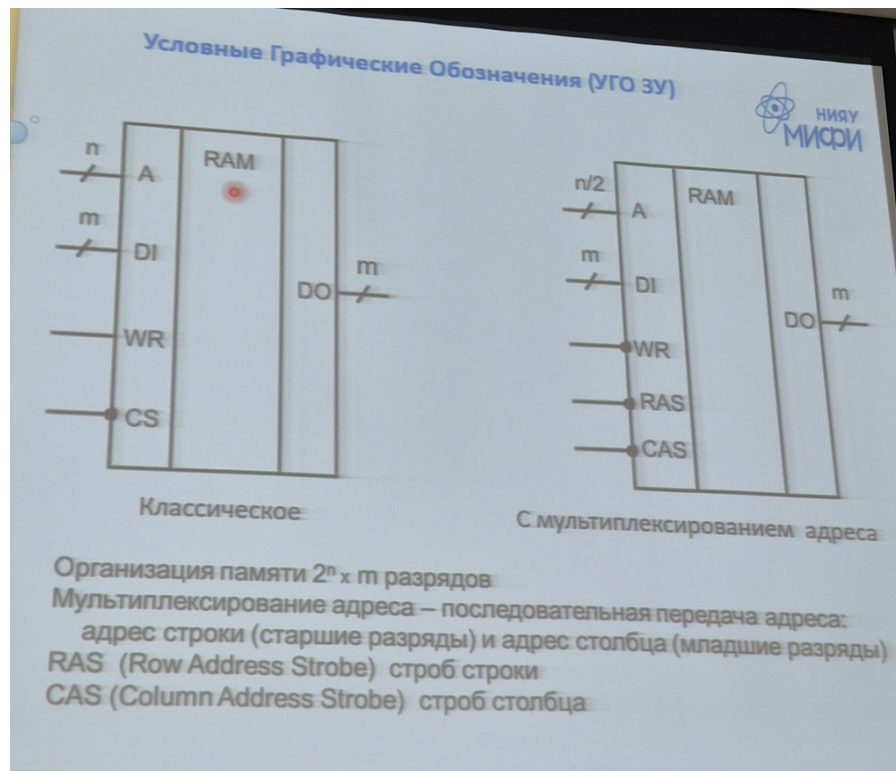
Время считывания — интервал времени между моментом сигнала на считывание (адрес или разрешение считывания) и моментом выдачи данных на выходы памяти.

Время записи - интервал времени после задания сигнала записи, достаточный для установления ячейки в состояние, задаваемое входными данными.

Цикл считывания/записи - минимальный интервал между последовательными обращениями.

2.5 Примеры условных графических обозначений (УГО ЗУ)

Запоминающее устройство:



n - разрядность шины адреса (количество проводов).

A - адрес.

DI - Data Input. m - размер слова этого запоминающего устройства.

WR - Write Read. Какую операцию выполнять

CS - Chip Select. Вход разрешения работы этого модуля. Если на схеме выколота точка, то это означает, что разрешающий сигнал "0".

Организация памяти: $2^n * m$

3 Семинар 2.

3.1 Вопросы (30.10.2025)

Из скольки частей состоит классическая ЭВМ? - из 3х

Какое волшебное слово объясняет, как эти 3 кубика работают? - циклически

Предназначение АЛУ? - выполнять АЛ операции. Что порождает АЛУ? - признак результата и результат.
 что делать - статический сигнал

когда делать - импульс

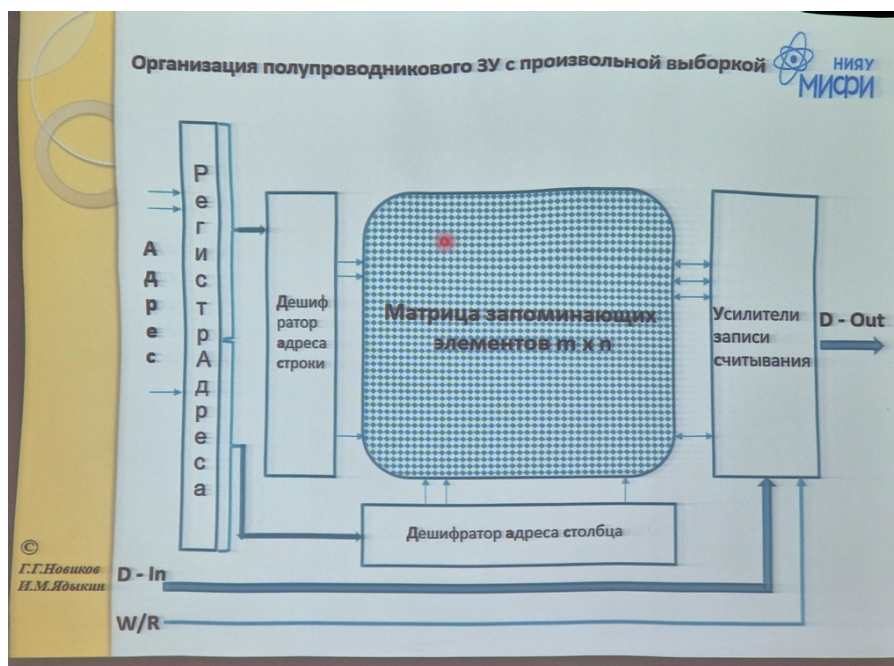
Адресуема только вся ячейка, так что любая операция происходит сразу со всей ячейкой.

3.2 Продолжаем УГО ЗУ

DI и DO абсолютно одинаковые

Если сигнал на CS не совпадает с разрешающим, то устройство находится в состоянии хранения.

3.3 Организация полупроводникового ЗУ с произвольной выборкой



Плоская двумерная матрица из запоминающих элементов. Бистабильные элементы.

Организовано так, чтобы можно было обратиться к любому элементу.

Дешифратор - электронное устройство, преобразующее позиционный двоичный код в код унитарный.

Унитарный - двоичный код, содержащий 1 единственную единицу. Значение определяется номером разряда, в котором находится единица.

Таким образом мы получаем указатель этой единицей на элемент строки или столбца.

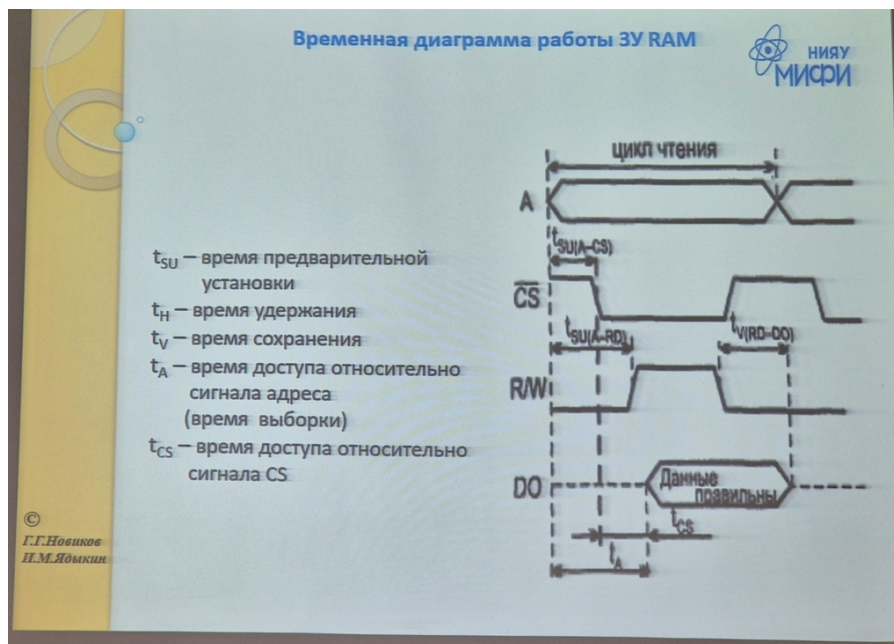
Регистр хранит 1 число - адрес, которое разделяется на 2 числа пополам, которые считаются номером адреса строки и номером адреса столбца.

3.3.1 Усилители записи считывания

Правильнее сказать: формирователи.

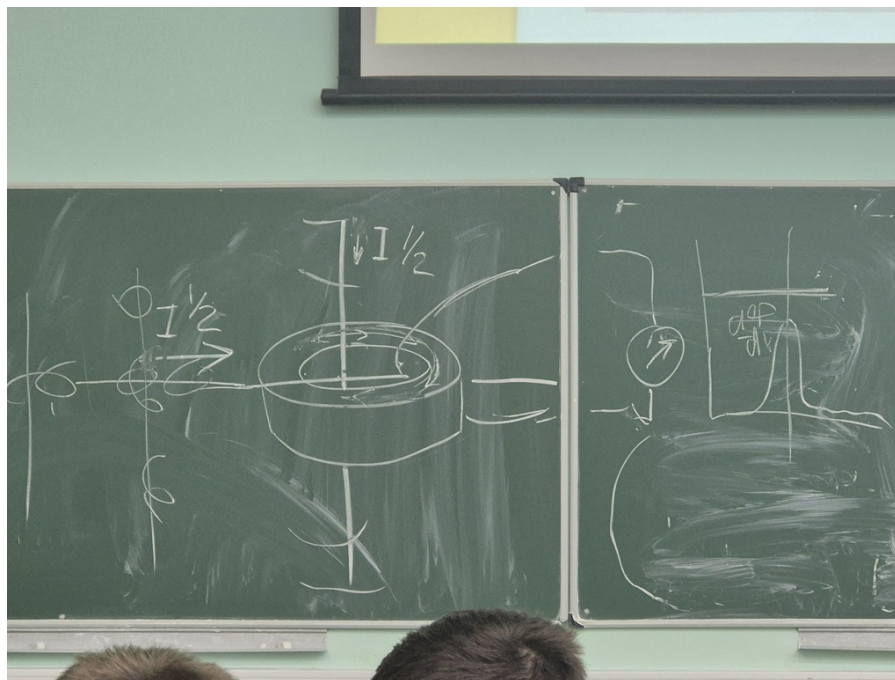
В зависимости от того, из чего сделаны элементы хранения могут быть разные потребности в вольтах и амперах. И суть этих устройств в согласовании внутренних сигналов и внешних стандартных логических сигналов.

3.4 Временная диаграмма работы ЗУ RAM.



Временная диаграмма - изображение на осях времени электрических сигналов несущих, значения логических переменных.

3.4.1 Запоминающее устройство на ферритовом сердечнике



Это сплав железа и чем-то ещё так, чтобы получилось хорошо (речь о магнитных свойствах) и испекли пирог в виде кольца. Это кольцо обладает способностью быть намагниченным в двух направлениях. Вокруг тока образуется по правилу буравчика направление намагничивания, а после того

Функции:

1. запись ... (всё те же 4)

Запись:

Мы пустим половину тока по 1 проволоке, а вторую половину по второй. (только полный ток перемагнитит). Там где токи пересекаются ячейка переманичивается, а другие кольца даже не почешутся.

Чтение:

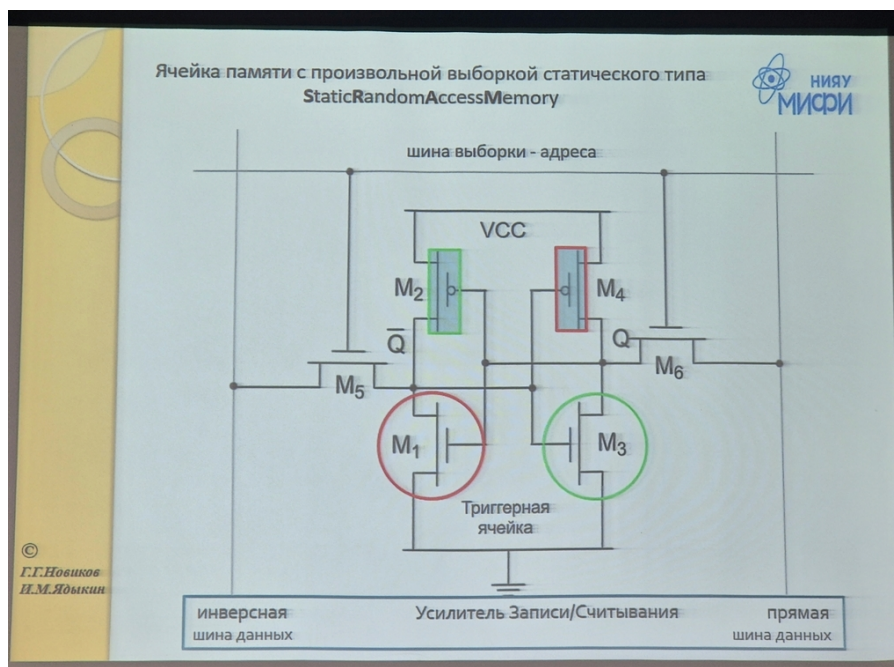
Пустим 3-ю проволочку (справа кривулька), не пуская по ней ток.

1. Без изменения намагничивания вольтметр ничего не покажет.
2. При изменении намагничивания возникнет импульс, вольтметр зафиксирует это.



Свойства: всё равно на радиацию. Разрушить можно только кувалдой.

3.5 Ячейка памяти с произвольной выборкой статического типа

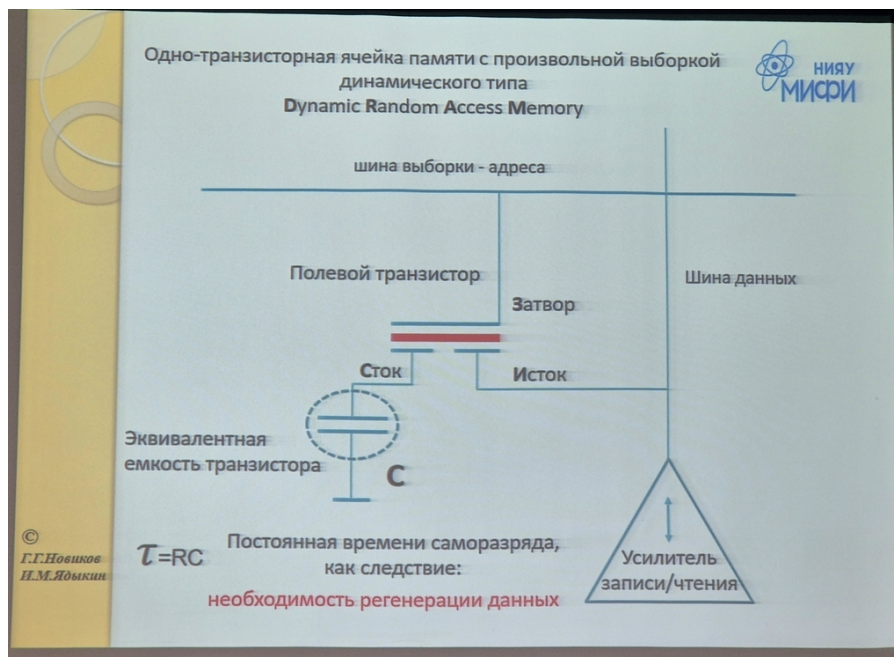


Пара ... выход одного соединён со входом другого. Исключает возможность прямого тока от шины питания к земле. Реализован на полевых транзисторах. Хранит только до тех пор, пока есть питание. Функции выборки: M5, M6. Они подключены к плечам ячейки и будучи открыты шиной выборки соединяют выбранную ячейку с усилителем, подавая туда 2 сигнала противоположной полярности.

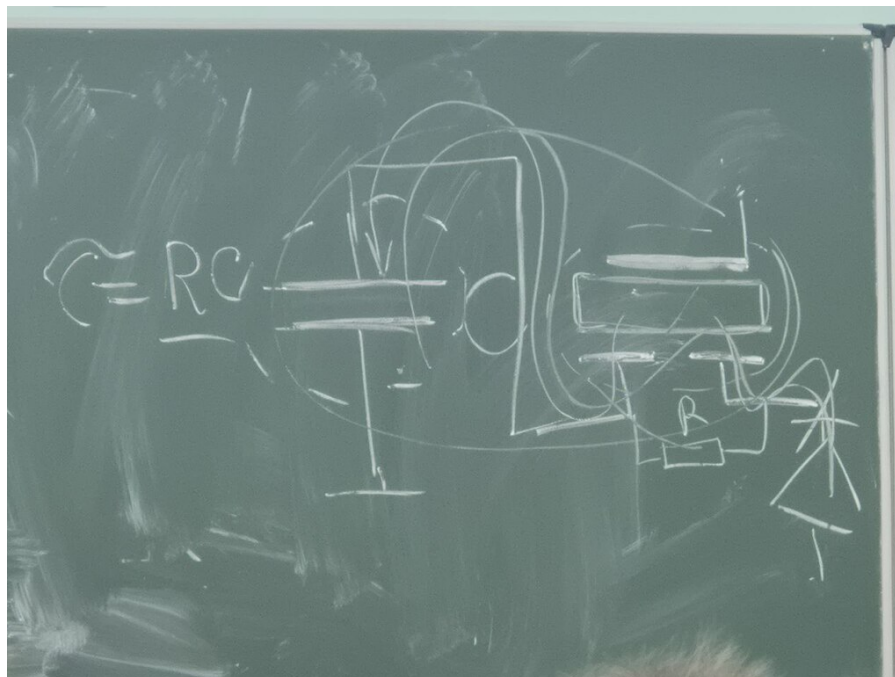
Зелёное и красное на слайдах меняется, поэтому биистабильно :)))

Здесь плохо, что ячейка хранит очень мало. 6 транзисторов на 1 бит.
Альтернативный вариант динамическое запоминающее устройство на полевом транзисторе.

3.6 Одно-транзисторная ячейка памяти с произвольной выборкой динамического типа.



Можно использовать конденсатор в качестве элемента хранения.



Эквивалентная схема с открытым транзистором и подключённый к усилителю, а затем закроем транзистор. Если опять открыть, то заряд потечёт в усилитель.

Но так как это полупроводник мы можем повесить дополнительное сопротивление и со временем заряд стечёт и информация будет потеряна. Чтобы этого не произошло нужно считать её, записать и делить это постоянно.

То есть хранение информации в такой ячейке происходит динамически и требует постоянного чтения и записи (регенерации информации). Почти 90% всем ЗУ устроены так.

Разреженная регенерация - регенерация не всего массива, а только тех, кто нужен. Любое обращение к ячейке вызывает регенерацию.

4 Лекция 10 от 06.11.2025.

Вопросики:

Чем отличается память от склероза? В том, что на экзамене от всё забудет :(

Какие 4 операции есть в ЗУ?

В каком виде ЗУ в процессе хранения необходимо слово выборка? - Регенирация памяти. Динамическая память.

Что такое регенирация? - чтение и обратная запись того, что прочли.

А каком виде ЗУ так же требуется записывать после простения? - В ЗУ на основе фееритового сердечника из-за разрушающего свойства.

В какой метрике среда хранения полупроводникового ЗУ с произвольной выборкой? - двумерный массив.

Что такое хорошо, а что такое плохо, когда речь о ячейке статического типа? Весь тот гемор с обращением динамически к памяти не нужен.

Чем больше ёмкость тем дольше он будет хранить в цикле регинерации.

4.1 АЛУ



Выполняет арифмитические и логические операции. Для арифмитических операций необходимо (по Нейману) иметь сумматор, но могут быть и дополнительные ускорители процессов вычисления. Минимальное число сумматоров, которые необходимы для работы ЭВМ - 1.

Для логических же операций необходимы:

- Логические элементы (минимально нуже базис)
- Дополнительное оборудывание, которое могло бы выбрать какую из операций необходимо выполнить.

Операции АЛ выполняются над **операндами** - число участвующее в А или Л опреации.

Схемма сумматора не обладают свойством сохранять свой состояние в отличие от ЗУ.

Комбинационная схема - схема значение которой есть только тогда, когда на вход поданы элементы.

Необходимо добаыть устройтсво для хранения операндом размером 1 (регистр). На схеме это ригистры операнда А и В.

Выходы регистров подключены к входам блока операций, а входы к тому месту где живут операнды (в памяти) соединяются магистралью с ЗУ.

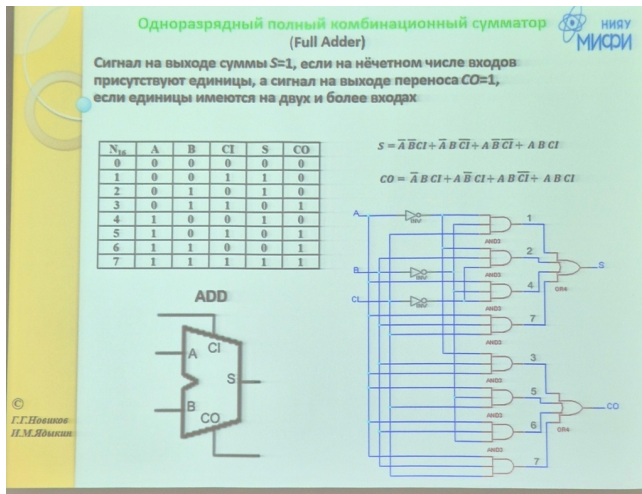
Результаты хранятся в памяти (записывается из регистра результата).

Дополнительная обратная связь между регистром результата и ... позволяет использовать результат в следующем вычислении. Появляется регистр аккумулятор (накапливающий).

Нужно указать АЛУ какую операцию выолнить. Это указание несёт в себе **коп** - код операции - число определяющее номер операции в списке возможных операций.

Признаки результатов не нарисовали и мстное устройство управления (в это не лезем).

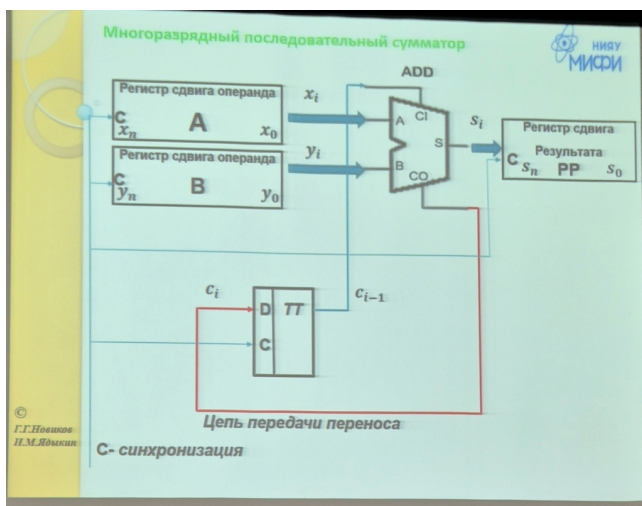
4.2 Одноразрядный полный комбинационный сумматор



последовательный сумматор.

Нужно сделать из него сумматор многоразрядный

4.3 Многоразрядный последовательный сумматор



стра.

PP - регистр результата.

Однако идея данного устройства в том, что это регистры обладающие функцией сдвига кода.

Картинка с нарушениями нужна только для демонстрации.

Сдвиги происходят одновременно во всех 3 регистрах из-за управляющего сигнала.

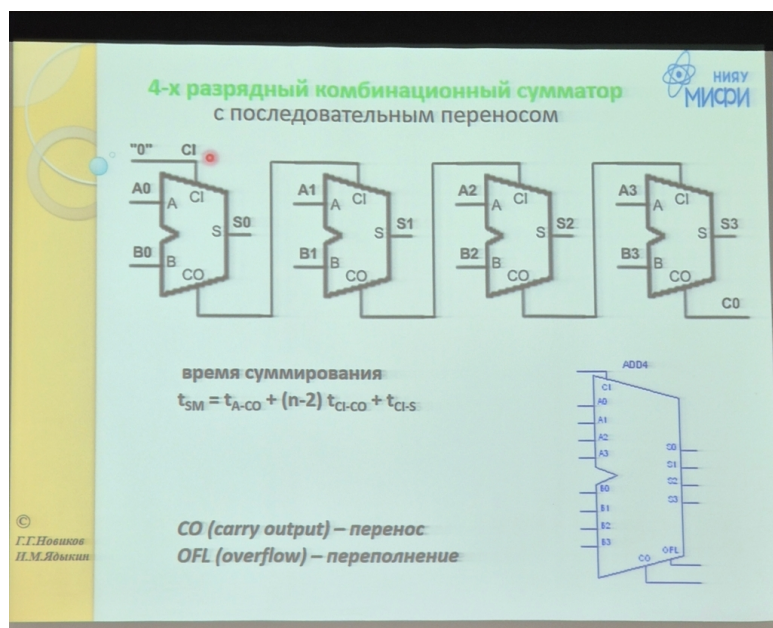
Триггер типа D передаёт в следующий такт то, что было на его входе в текущем (D - delay).

Все имеют вход с буквой C - вход для синхронизации. Подаётся сигнал синхронизации на все элементы которые должны выполнить действия одновременно.

Хорошо: объём оборудования весьма скромный (1 комбинационный сумматор и несколько регистров с функцией сдвига + триггер).

Плохо: Время выполнения суммирования прямо зависит от количества операндов (8 тактов для байта).

4.4 4-х разрядный комбинационный сумматор.



Это уже кусочек схемы. Входы слева, выходы справа. Логическое распространение сигналов слева направо, сверху вниз. Для каждой пары разрядов свой собственный отдельный одноразрядный комбинационный сумматор. Так что вычисление суммы происходит одновременно за 1 такт.

Плохо: с переносом проблема.

5 Устройство управления

Что делает? - Пораждает сигналы управляющие.

Из чего порождает? - Исходя из программы.

Программа - алгоритм решения задачи, записанный с помощью команд ЭВМ.

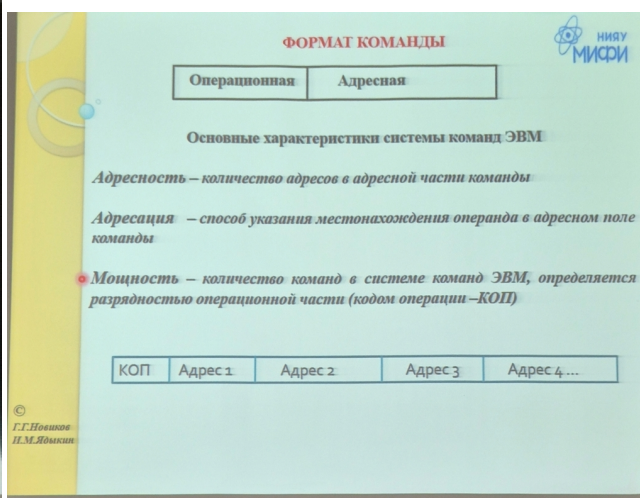
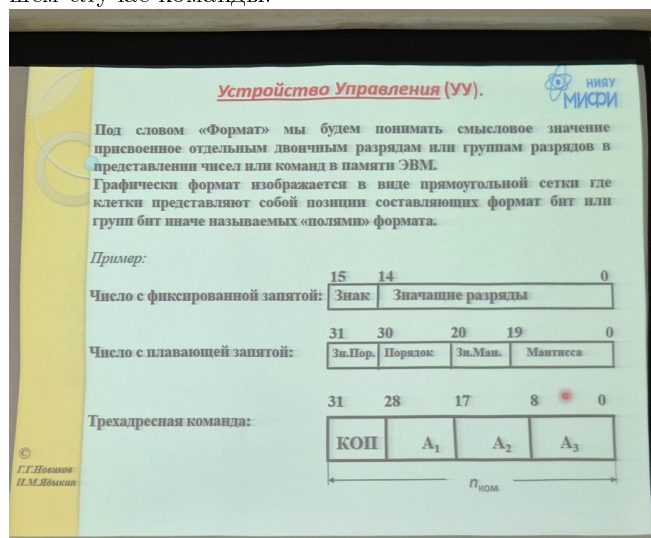
Алгоритм - последовательность действий необходимые для решения задачи.

Устройство Управления формирует управляющие сигналы исходя из того, какую команду оно выполняет.

Суть - выполнение команды программы.

Команда программы - число.

Что такое формат? - смысловое значение присвоенное отдельным разрядам в двоичной записи. В нашем случае команды.



1. Операционная часть
2. Адресная часть

Всякая команда состоящая из этих частей имеет некоторый набор характеристик. Операционная часть команды содержит в себе:

- КОП (Код ОПерации - двоичное число идентифицирующее команду в системе команд ЭВМ)

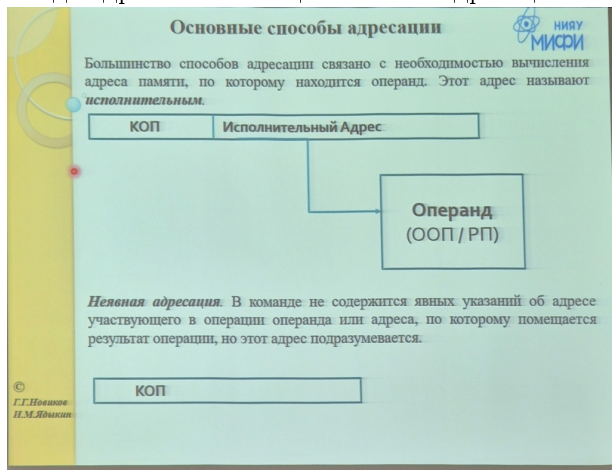
Какая связь между кодом операции и количество разрядов кода операции: 2^n - Мощность. Адресная часть:

1. Адреса операндов, участвующих в этой команде.

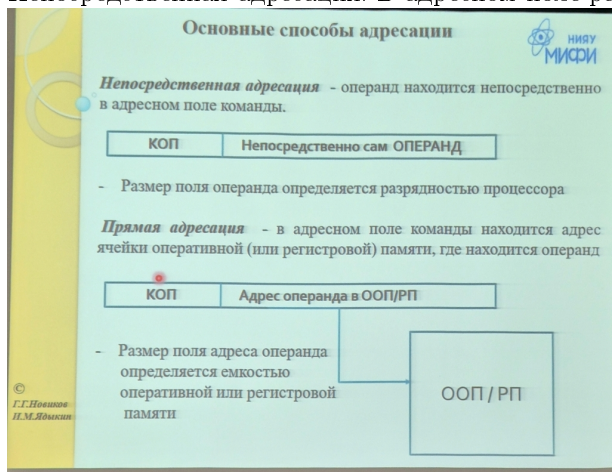
Адресация - способ указания местоположения этого операнда в адресном поле команды. Способов этих можно придумать бесконечно.

Основные способы адресации:

1. Когда адреса нет вообще. Неявная адресация. Нет указания куда помещать результат.

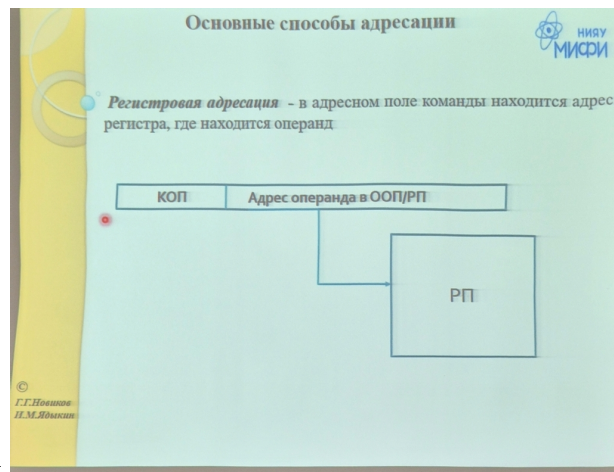


2. Непосредственная адресация. В адресном поле располагается сам операнд.



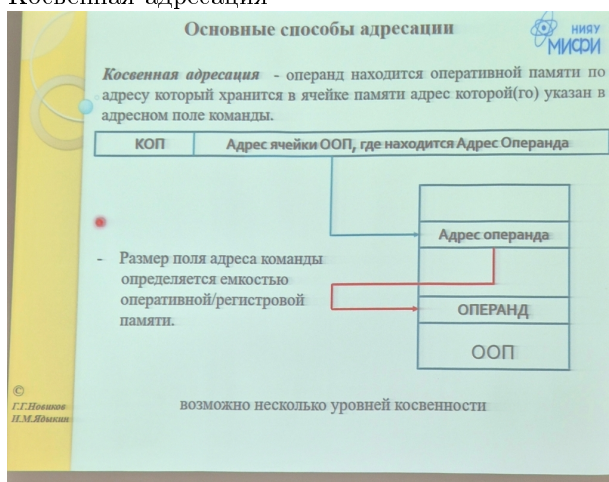
Чем команда меньше, тем быстрее она будет извлекаться.

3. Прямая адресация.



4. Регистровая адресация

5. Косвенная адресация



Адрес адреса операнда. Может быть построена на нескольких уровнях вложенности.

6 Лекция 11. От 13.11.2025

6.1 Вопросы

Что такое формат команд? - 2 части: операционная и адресная. В операционной - код операции (двоичное число, которое однозначно идентифицирует команду в системе команд)

С какой характеристикой ЭВМ связана величина поля ввода операции? - Мощности системы команд эвм, что равно количеству команд в штуках.

Какая характеристика позволяет оценить количество полей в адресной части команды? - количество адресов (адресность)

Как называется то, что определяет способ записи местонахождения операнда в этом адресном поле - адресация или способ адресации.

Где следует искать операнд если использована непосредственная адресация - непосредственно в этом адресном поле.

А если адресация прямая - искать в оперативной памяти по адресной ячейки, который написан в адресном поле команды.

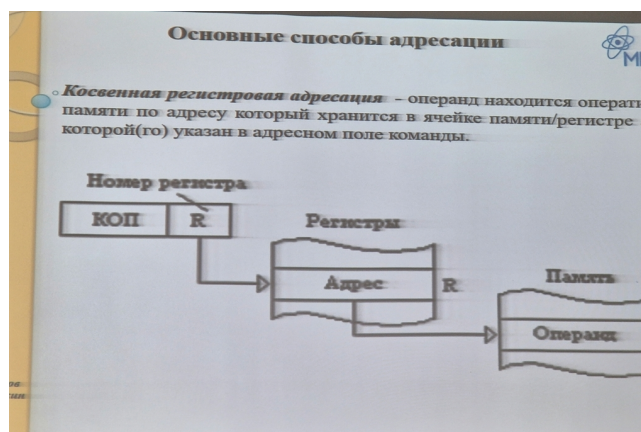
А если косвенная - искать адрес операнда в ячейке адрес которой указан в адресном поле команды. Адрес адреса операнда.

Поругайте мне косвенную адресацию - многократное обращение в памяти (время докапывания до операнда (минимум 2 раза)).

О какой характеристики ЭВМ можно судить о величине адресного поля при использовании косвенной адресации - по объёму оперативной памяти.

Как сохранить косвенность и справиться с проблемами косвенной адресации? - использование варианта косвенной адресации, в котором хранится информация не в ячейке ОП, а в регистре, получается косвенная регистровая адресация.

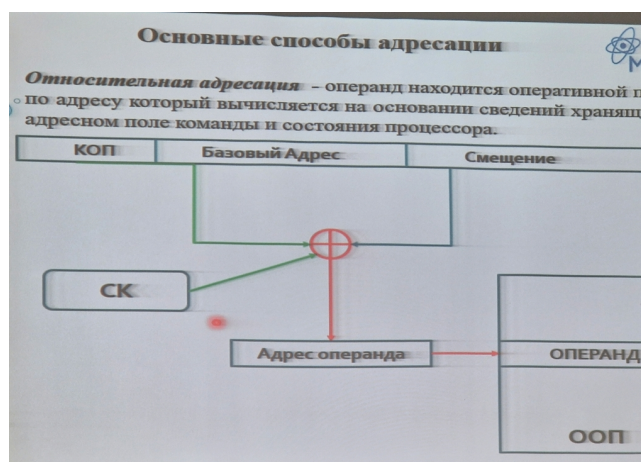
6.2 Регистровая адресация



Скорость наивысшая.

1 обращение в регистровую память, 2 обращения по адресу, извлечённому из регистра обращаемся в ООП. Получаем все плюшки косвенной адресации.

6.3 относительная адресация

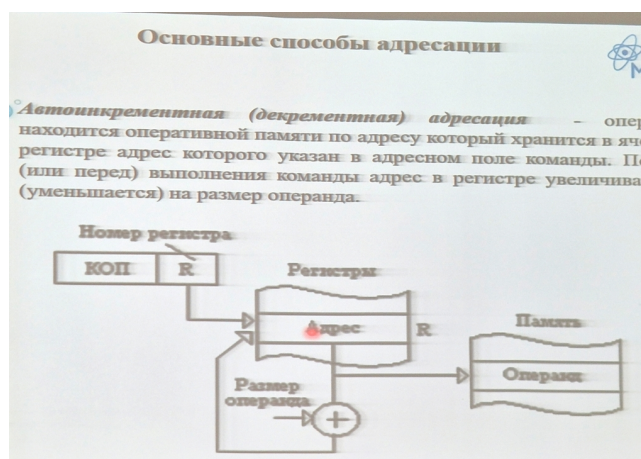


Местоположение операндов в памяти задаётся с помощью смещения относительно чего-то. Обычно относительно некоторого базового адреса. Т.о. получается, что при формировании адреса операнда используется суммирование, а именно сумма адреса базового и смещение.

Крестик на фото и есть суммирование.

СК - счётчик команд - хранит следующую команду. Местонахождение операнда определяется относительно местонахождением команды. В момент, когда надо загрузить команду в память задаётся начальное смещение в виде точки, куда будет загружаться программа - классика при мультипрограммном режиме.

6.4 Автоинкрементная адресация

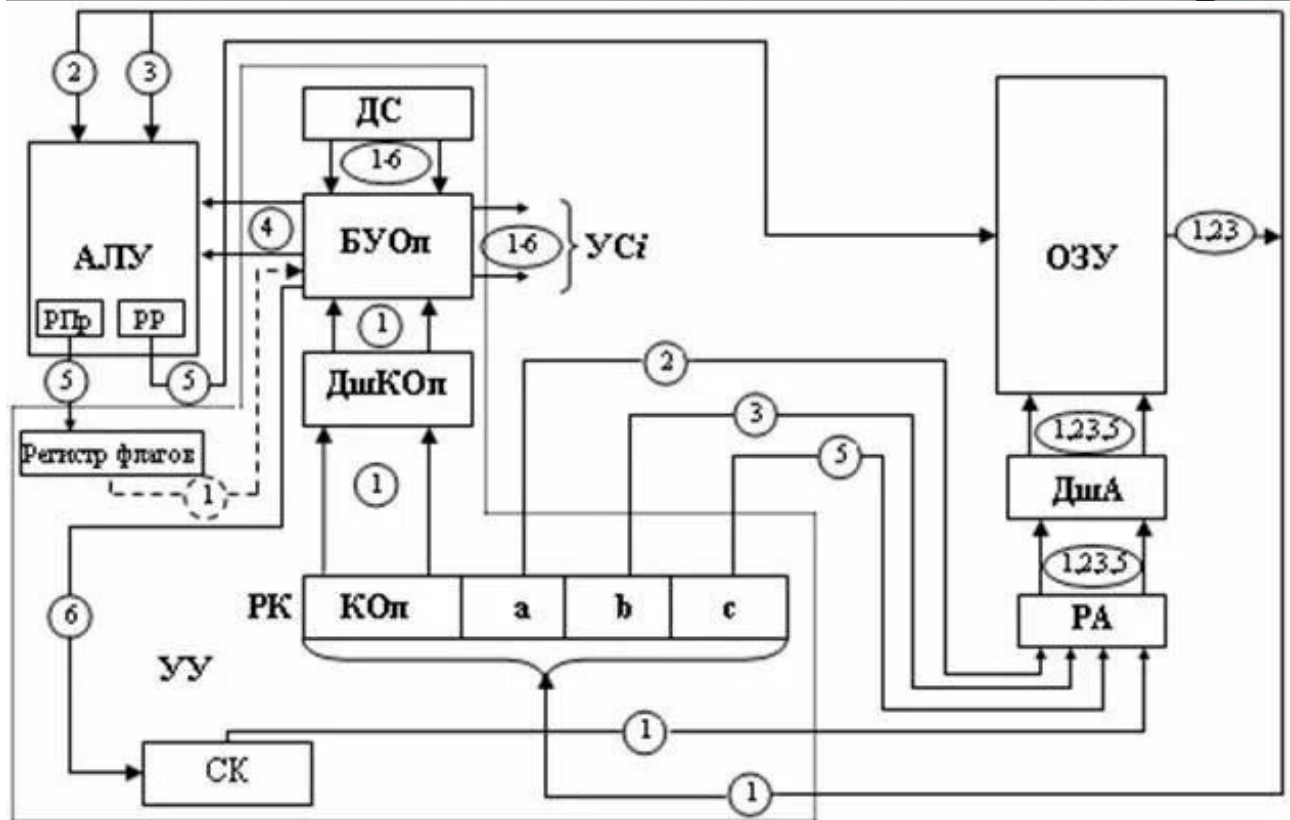
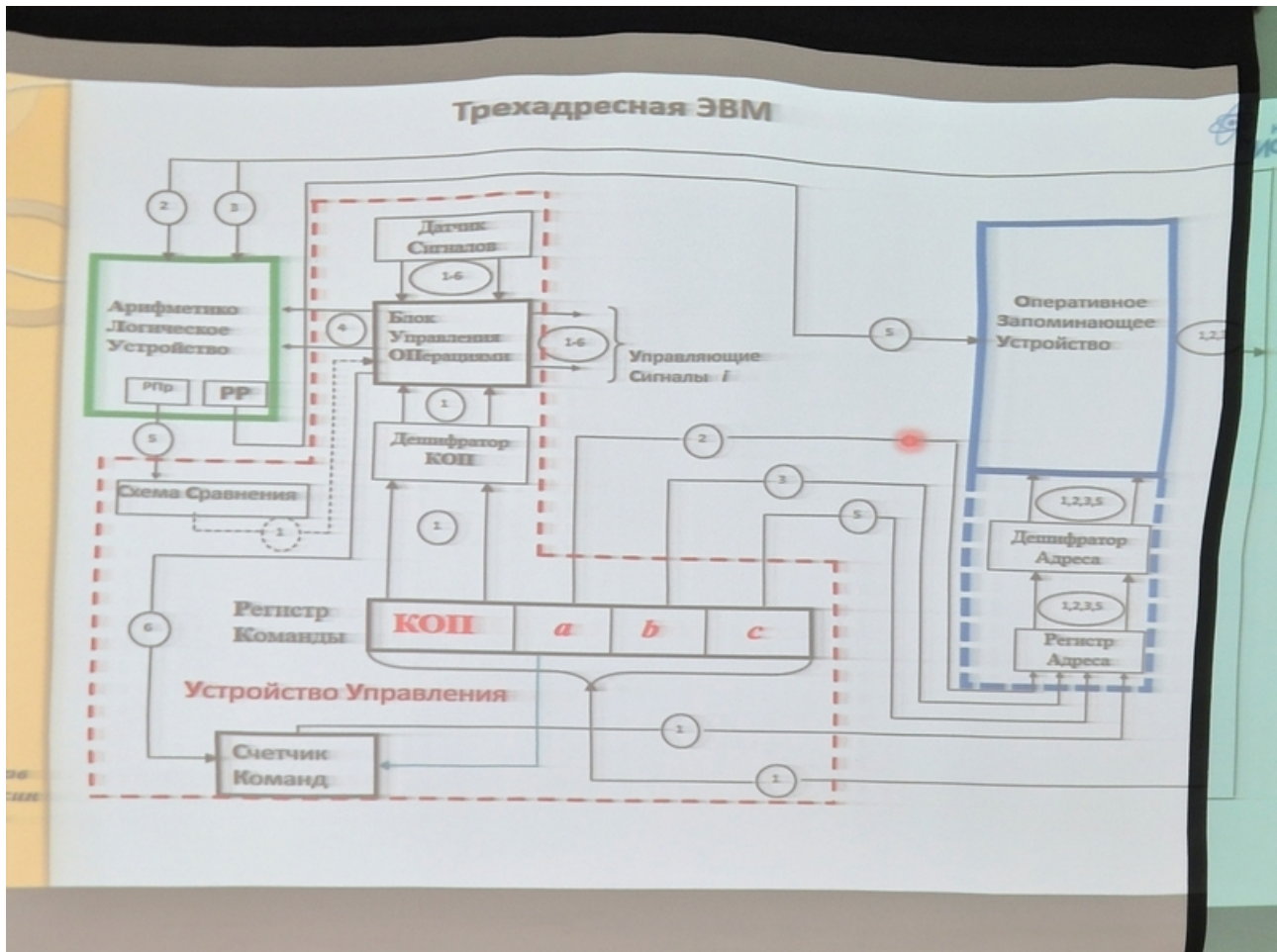


Не что иное как косвенная/косвенная регистровая адресация, у которой адрес операнда может меняться путём увеличения инкремента на константу при каждом использовании этой команды. Т.о. при запуске такой команды в цикле получим последовательное обращение подряд идущих элементов (нужно например для массивов). Существует в варианте инкремента и декримента - одно увеличивает, другое уменьшает.

6.5 Заключение по видам адресаций

Адресаций может быть бесконечно много. Нужно знать неявную, прямую, косвенную, регистровую, косвенную регистровую, автоинкремент декримент, относительную адресации.

6.6 Трёхадресная ЭВМ



Состоит из:

1. Арифметико-Логическое Устройство. В нём логические элементы. На вход принимает операнды и

пораждается результат (РР) и признак результата (РПр).

2. Оперативное Запоминающее Устройство. Функции: хранение, чтение, запись, выборка.

3. Устройство Управления.

Оно разделяется на 2 основные части:

(а) РК - Регистр команды. Обращаем внимание на то, что речь про 1 и только 1 команду.

Число, которое хранится в этом регистре представляет собой команду. Формат этого числа - формат команды.

Структура регистра команды определяется опаратной реализацией формата команды. Число записанное туда как раз и занимает поля формата команды и содержит всё что там полагается. Могут обитать 3 адреса: a, b, c, а также сам Код ОПерации.

Команд много, а регистр может хранить 1 единственную программу - выполняемую в данный момент времени. Именно по этой причине **Функция регистра команды**: хранение команды в процессе её выполнения (той самой 1 единственной).

Логичные вопросы, которые возникают:

- Как команда попадает в регист команды?
- Как команды в памяти превращаются в программу?

Программа - алгоритм для решения задачи, записанный командами ЭВМ.

Способ адресации команд программы (не путать со способами адресации операндов) по сути способ связывании команд в программу:

i. **Принудительная адресация программ команд программы.**

Суть заключается в том, что команда в своей адресной части имеет адресное поле с адресом следующей команды (обычно где-то в конце). Получается, что текущая команда выполняемая в данный момент несёт в себе адрес следующей программы. Получается структура, которая называется **связанный список** (блок-чайн без шифрования ;)).

Плюсы и минусы такого способа:

- Никаких усилий для вычисления адреса следующей программы не нужно. команды могут располагаться как угодно в любых ячейках памяти.
- Занимает место. Приходится тащить поле размером с адрес основной оперативной памяти, а команд много, так что количество необходимой памяти сильно увеличивается.

ii. **Естественная адресация команд программы.**

Берётся память (одномерный массив ячеек или же вектор) и в неё последовательно в том порядке, в котором они должны выполняться вносятся команды. При таком способе к конструкции добавляется устройство, хранящее адрес команды и обладающее возможностью инкремента для добавления некоторой Δ , что является размером данной команды, - это называется **счётчик команд**.

Плюсы и минусы такого способа:

- Избавляемся от поля для команды, значит не приходится тратить лишнюю память.
- Нужно класть команды в ячейки строго в порядке выполнения и необходимо потратить дополнительное оборудование для формирования адреса следующей команды + небольшое время добавления дельты к адресу.

Преимущество в размере команды превалирует над первым способом, поэтому обычно используется естественная адресация.

В трёхадресной ЭВМ используется именно естественная адресация (видно счётчик команд ;))

Счётчик команд - Instruction Pointer - IP - указатель на команду.

ОЧЕНЬ ВАЖНО! Никакой связи IP с интернет протоколами на зачёте нет.

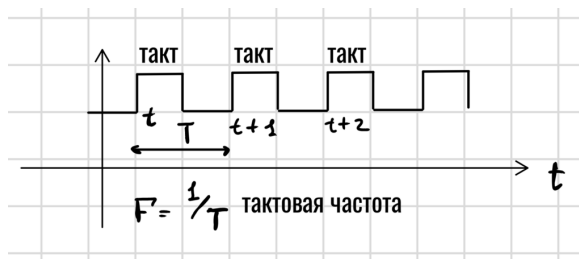
Счётчик команд передаёт адрес следующей команды на Регистр Адреса ЗУ, указывая на ячейку памяти, где она хранится и считанное по этому адресу число загружается в Регистр Команды.

(b) Управление сигналами:

Помним, что Устройство Управления решает 2 задачи: что делать и когда это делать.

Что делать понятно из команды, расположенной в регистре команд, код операции которой передаётся в дешифратор кода операции. Смысл этого действия в том, чтобы сообщить той части, которая формирует сигналы (Блоку Управления ОПерациями), какая это команда.

Вопросом коагда занимается Датчик Сигнала (служба времени было-бы красивее @Новиков).



Что такое время с точки зрения физики?

Время - это физическая величина, которую не определить, но оно обладает свойствами: время течёт равномерно и непрерывно.

Заменяем физическое время на **время дискретное**.

Для этого мы используем генератор квантов (сигналов) и представляем время при помощи импульсов.

Импульс - изменение во времени состояние электрического сигнала несущего значение логической переменной из исходного состояния в противоположное.

Пусть 1 импульс будет равен 1 **кванту времени** или же по другому 1 **такту**, тогда другой импульс будет другим тактом и т.д.

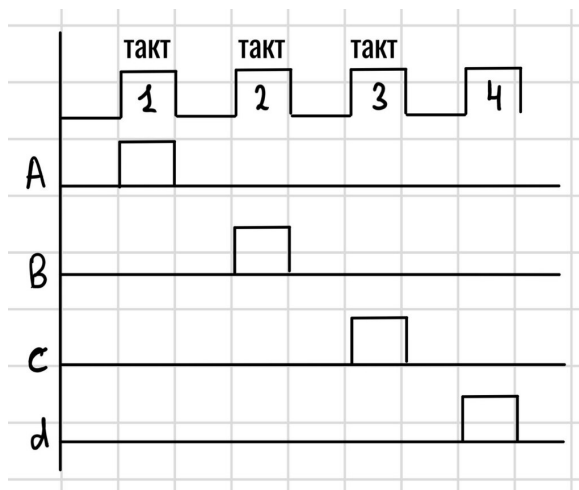
А между импульсами ничего нет. Тогда в нашем новом дискретном времени все события происходят в этих тактах. Поэтому эти такты мы можем пронумеровать (t - сейчас, $t+1$ - следующий момент времени, $t+2$, ...). Можно и в обратную сторону ($t-1$, $t-2$, ...).

Порадив такое квантовое время и поняв, что события происходят только в тактовый импульс мы становимся хозяевами того, что происходит в каждый конкретный импульс. Но физическое время никуда не делось и в нём происходят импульсы, а раз они формируются в физическом времени, то у них есть тот же набор характеристик, что и у импульсного сигнала:

- Длительность
- Амплитуда
- Фронт
- Период (T)- время между одинаковыми частями 2 соседних импульсов.
- Частота (тактовая частота) - величина обратная периоду: $F = \frac{1}{T}$. Её порождает генератор тактовых импульсов.

Изменяя частоту мы изменяем насколько часто будут происходить события в нашем устройстве. Чем быстрее тактовая частота тем быстрее работает устройство.

Мы должны иметь возможность внутри цикла выполнения команды указывать на моменты времени, в которые должны выполняться те самые действия, которые будут повторяться. Т.о. мы должны не только разбить время на кванты, но и иметь возможность выбирать (наверное тут было: какой квант выполнять). В этом помогает **распределитель импульсов цикла**.



В этом примере мы получили, что у нас есть 4 сигнала, которые связаны с друг с другом так, что импульсы на тактах 1, 2, 3, 4 будут соответствовать ...

Подавая с линии А выполняется 1 такт, а на линии С - 3 такта.

Схематехнически это выглядит так:



Вот тут у меня плохо получилось:

Каждый тактовый импульс изменяет состояние счётчика на +1. Дешифратор преобразует позиционный код в унитарный.

Пришёл импульс, счётчик переключился, ообразовался один ноль (единица двоичная) единица на дешифраторе переместилась на 1 выход. След на дешифратор, счётчик на 2. 1 не выходе дешифратора переместилась на выход номер 2. Т.о. прошёл всё да в пизду.

Дешифратор кода определяет какие именно импульсы нужны для реализации команды. Объединяет всё блок управления операциями (что и когда). Из него во все стороны и формируются i-е управляющие сигналы, подключающиеся к устройствам.

Вопросики:

Какие времена существуют: физическое и дискретное.

Какое время в ЭВМ: дискретное время.

Какая связь дискретного и физического времени? Дискретное время - время физическое разрубленное на кванты.

Что происходит между t и t_1 ? Между t и t_1 как раз и происходят все процессы в ЭВМ.

Как связаны переходные процессы с интервалом физического времени между t и $t+1$... - тактовые импульсы - импульс, которые продвигает дискретное время к $t+1$. Какая характеристика определяет что-то там

- Период периодического тактового сигнала. Обратная ей - частота Ограничение для увеличения частоты

- те самые переходные процессы. Если сильно уменьшить интервал t и $t+1$, так что логические синаои не успеют совершить свой переход, то всё поломается. Именно это определяет максимальную частоту. Как

работает классическая ЭВМ? - Циклически - посторение 1 и того же набора действий.

ЭВМ повторяет тот набор действий, которыеей необходим для выполнения команды - цикл выполнения команд.

ЗадресЭВМ - значит, что она имеет в своей части 3 адреса поля. Как связаны формат команды и РК - неразрывно, тк формат команды и регистр команды обсолютно одинаковы. Ведь оказавшись записанно в этот регистр тот, число записанное в памяти обретаает смысл его формата. Счётчик команды - хранит адрес следующей команды и формирует адрес следующей команды путём инкремента на длинну текущей команды. а) хранит адрес следующей команды б) формирует его. Кешифратор КОП - определяет какой набор сигналов необходим для выполнения текущей команды. Датчик сигналов - поставляет тактовые импульсы в развёрнутой картине (развёртка во времени событий происходящих внутри цикла выполнения команды) Блок управление операции - взяв с одной стороны что сделать с другой стороны в какие момены надо это сделать, соединяет одно с другим и порождает управляющие сигналы.

Структура регистра команд:

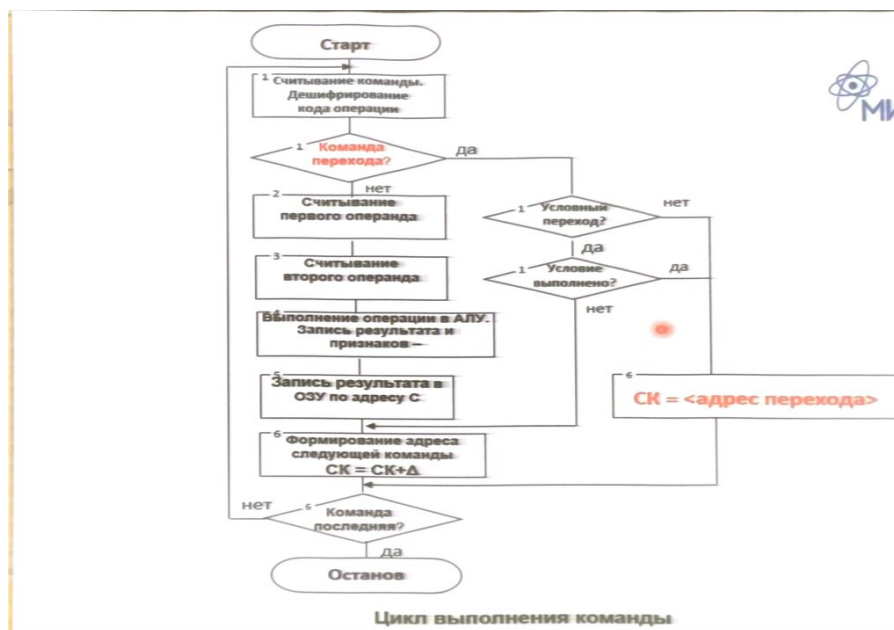
- Код операции - код операции команды

- a - адрес первого операнда.
- b - адрес второго операнда.
- c - адрес куда записать результат.

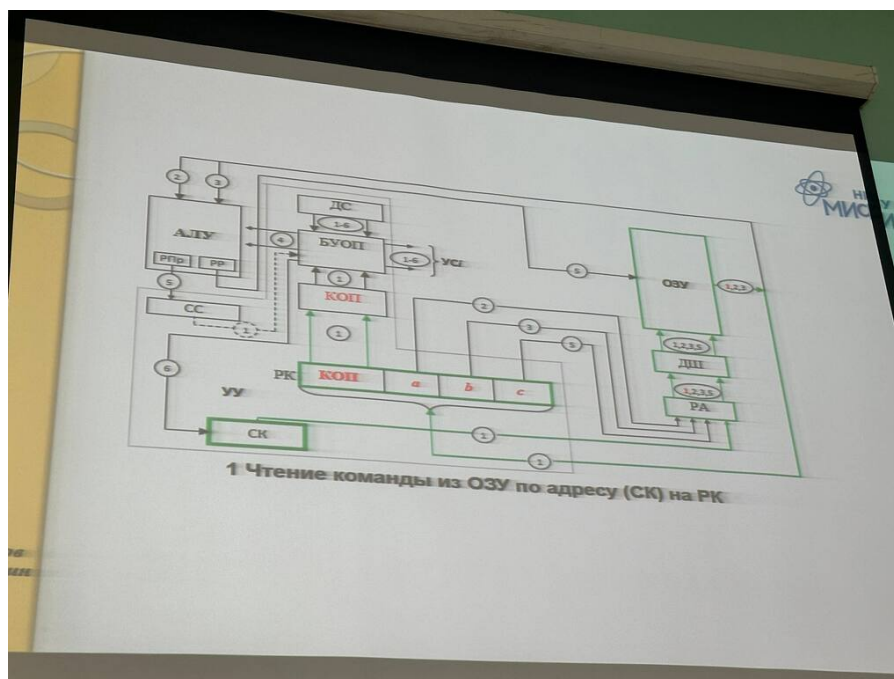
7 20.11.2025 Вычислительный процесс в классической трёхадресной ЭВМ.

Начальное условие и предположения:

- программа и операнды находятся в оперативном запоминающем устройстве (ОЗУ)
- адрес ячейки ОЗУ, в которой находится выполняемая команда, находится в счётчике команд
- команда считывается за одно обращение к ОЗУ;
- команда, операнды и результат используют прямую адресацию к памяти.

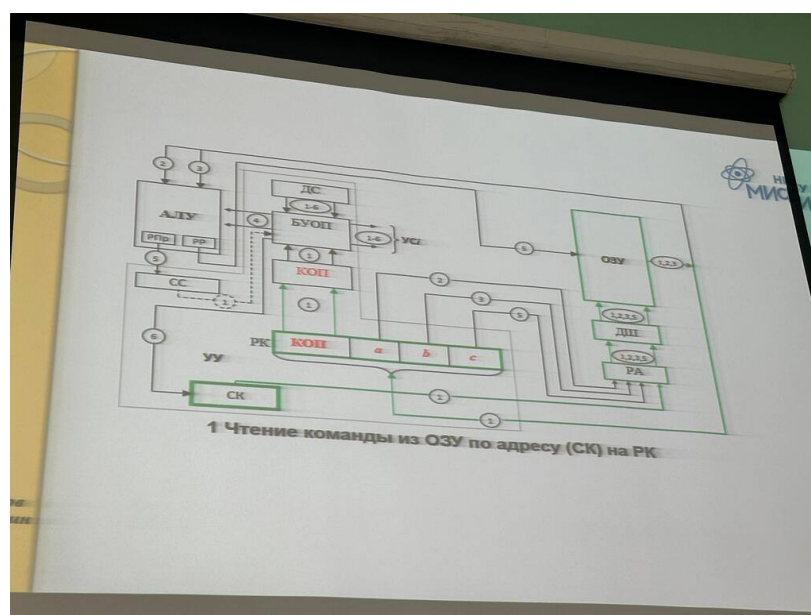


Линейная команда - команда, которая выполняет преобразования данных операндов. Команда ветвления - никак не изменяют данные и используются исключительно для изменения выполнения порядка исполнения команд программы.



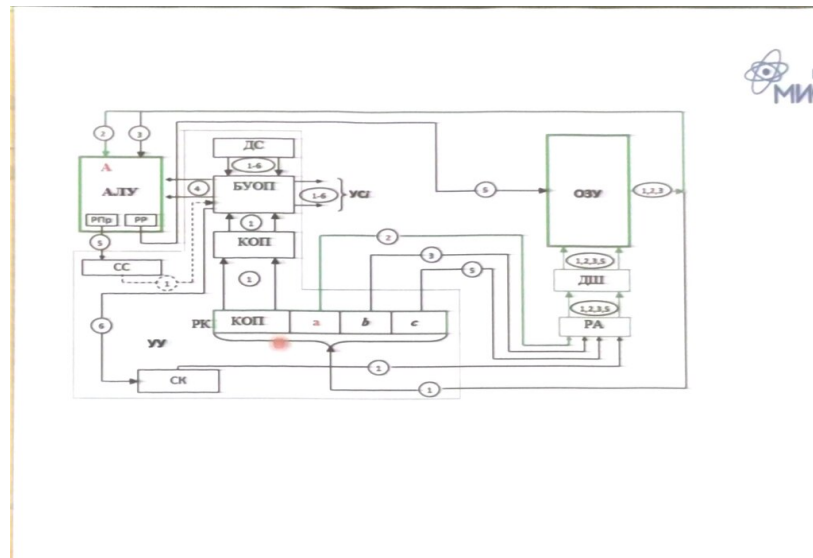
1. Выполнение линейной команды

(а) Этап 1. Выборка команды

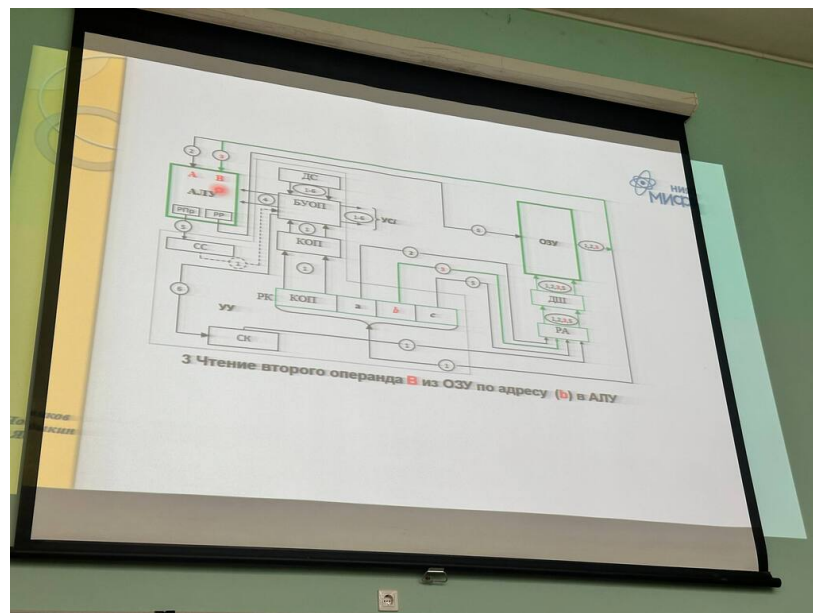


- Из счётчика команд число (адрес текущей команды) передаётся на вход памяти — в её регистр адреса. Внутри ОЗУ этот адрес дешифрируется, и из соответствующей ячейки извлекается команда. Её содержимое по первой магистрали передаётся в управляющее устройство (УУ), а именно — в регистр команды. Попав в регистр команды, это число интерпретируется как команда в соответствии с её форматом. Поле кода операции (КОП) направляется в дешифратор кода операции, чтобы определить, какая именно команда была считана из памяти.
- **Кратко:** Чтение команды из ОЗУ по адресу, хранящемуся в счётчике команд.
- Определено: команда линейная, в формате трёхадресной ЭВМ — содержит три адресных поля: два операнда-источника (А и В) и один адрес результата (С).

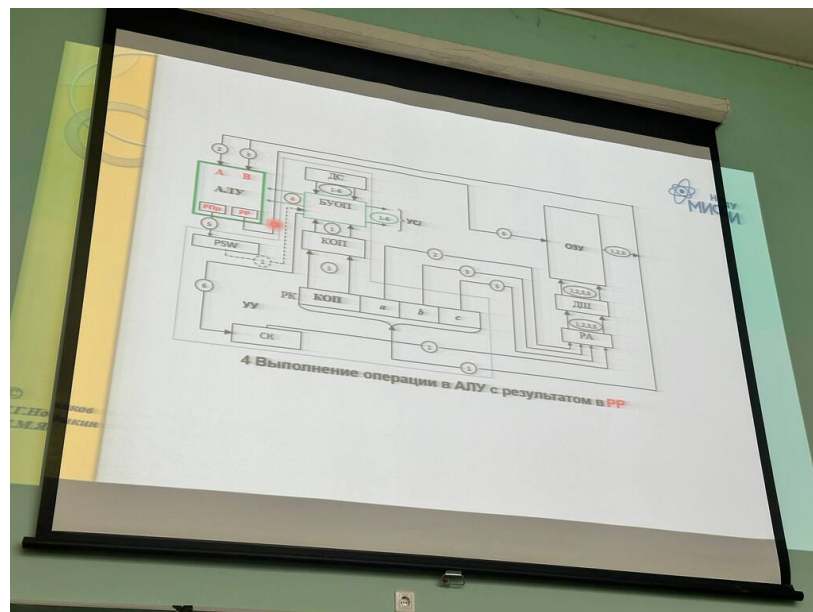
(b) Этап 2. Чтение первого операнда (А)



- Из поля А регистра команды адрес передаётся в регистр адреса ОЗУ. По этому адресу выбирается ячейка, и её содержимое (первый операнд) поступает на выход памяти. Операнд передаётся по второй магистрали в АЛУ — в регистр первого операнда (регистр А).
- **Кратко:** Чтение первого операнда из ОЗУ по адресу А и передача его в АЛУ.



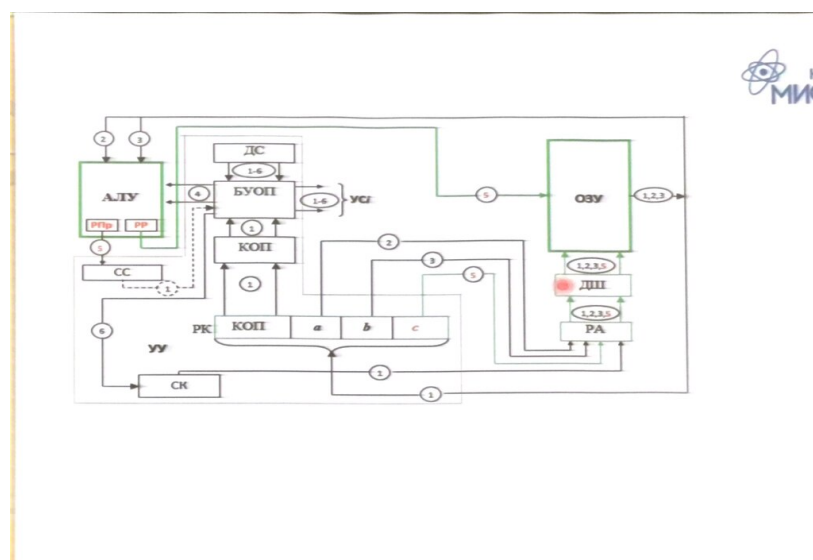
- Из поля В регистра команды извлекается адрес, который передаётся в ОЗУ. Выбирается соответствующая ячейка, её содержимое интерпретируется как второй операнд и передаётся в АЛУ — в регистр операнда В.
- **Кратко:** Чтение второго операнда из ОЗУ по адресу В и передача его в АЛУ.



- АЛУ выполняет операцию над двумя операндами. Полученный результат записывается в регистр результата, а признак результата (например, ноль, переполнение, перенос и т.д.) — в регистр признаков результата.
- **Кратко:** Выполнение операции в АЛУ; результат и его признаки сохраняются в регистрах.

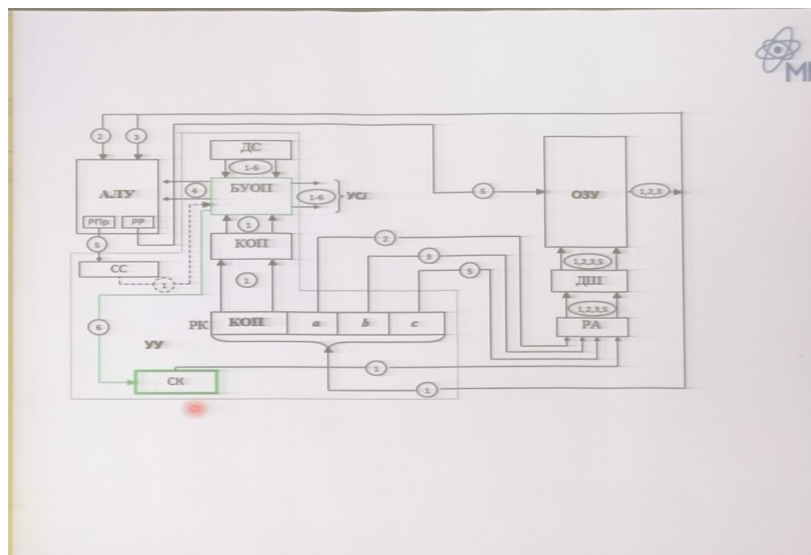
Скорее всего Новиков должен был рассказать тут и про формирование управляющих сигналов, но у меня этого нет. Кажется этого вообще не было.

(e) **Этап 5. Запись результата**



- Из поля С регистра команды извлекается адрес, который передаётся в регистр адреса ОЗУ и дешифрируется. Одновременно результат из регистра результата передаётся по информационной магистрали на вход записи ОЗУ. По приходу управляющего сигнала записи результат сохраняется в ячейке памяти по адресу С.
- **Кратко:** Запись результата в ОЗУ.

(f) **Этап 6. Обновление счётчика команд**



- Содержимое счётчика команд увеличивается на длину текущей команды (в данном случае — на 1). Теперь счётчик указывает на следующую команду в памяти.
- **Кратко:** инкремент счётчика команды.

На это цикл заканчивается и всё начинается заново.

Особенности:

- В системе **нет механизма**, позволяющего определить конец программы.
- Цикл выполнения **не прерывается**, пока включено питание.
- Команды остановки не существует.**
- Машина представляет собой автомат, предназначенный **только для бесконечного выполнения этого цикла.**

Примечание: описанный выше цикл справедлив **только для линейных команд.**

2. Выполнение команды перехода В трёхадресной ЭВМ существует **второй тип команд** — команды перехода.

(a) Этап 1. Выборка команды

(Полностью идентичен этапу 1 для линейной команды.)

(b) Этап 2 (не факт). Распознавание команды перехода

- Если дешифратор определяет, что команда — **команда перехода**, её формат интерпретируется иначе:
- Помимо КОП, используется **одно адресное поле (А)** — **адрес перехода**. Подразумевается, что по этому адресу находится команда, с которой должно продолжиться выполнение. Далее возможны два варианта:

i. Безусловный переход

Адрес из поля А **непосредственно загружается в счётчик команд**, замещая его текущее значение. На следующем цикле выборки команда будет считана уже **по новому адресу**.

ii. Условный переход

Команда содержит **два поля**:

А — адрес перехода,

В — **маска** (шаблон значений регистра признаков результата).

Процесс:

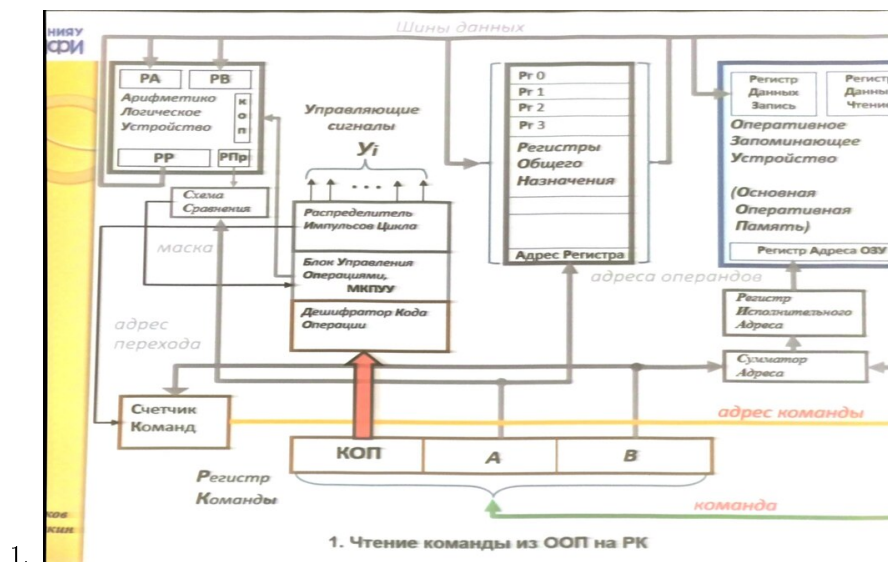
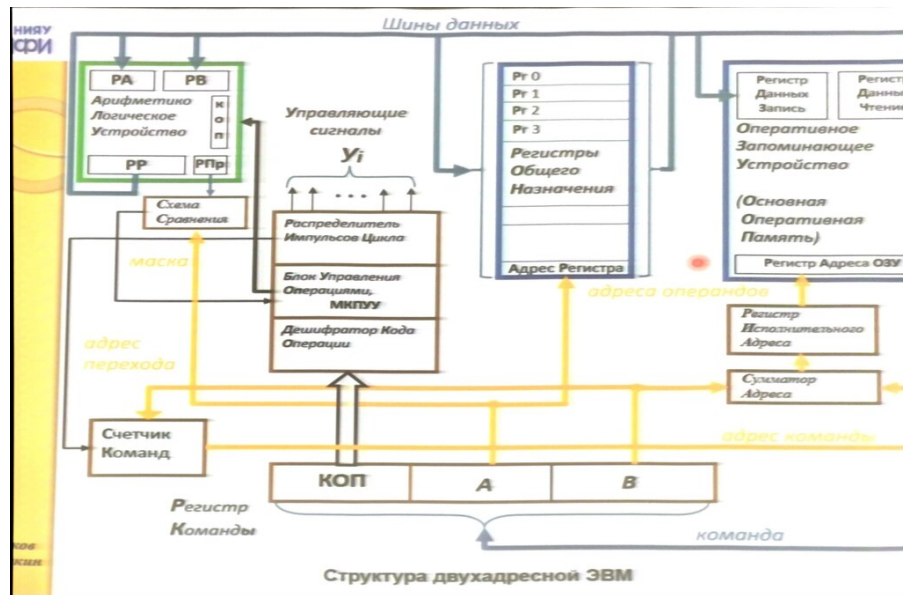
- Маска передаётся на схему сравнения.
- В схеме сравниваются текущие значения **флагов** (признаков результата) с маской.
- Если совпадение есть** — адрес А загружается в счётчик команд.
- Если совпадения нет** — счётчик команд инкрементируется на 1 (как при линейной команде), и выполнение продолжается со следующей команды.

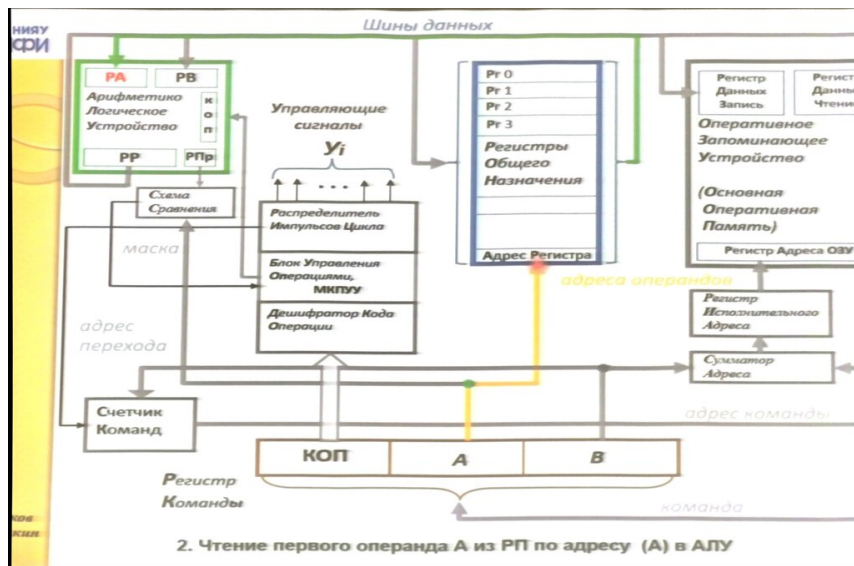
3. Что хорошо:

- Машина **работает** и полноценно функциональна.
- **Что следует учитывать:**
- Отсутствие команды останова требует явного управления потоком программы (например, бесконечным циклом или переходом на начало).
- Все поведенческие особенности определяются **форматом команд** и логикой дешифрации КОП.

8 Двухадресная ЭВМ

Структура

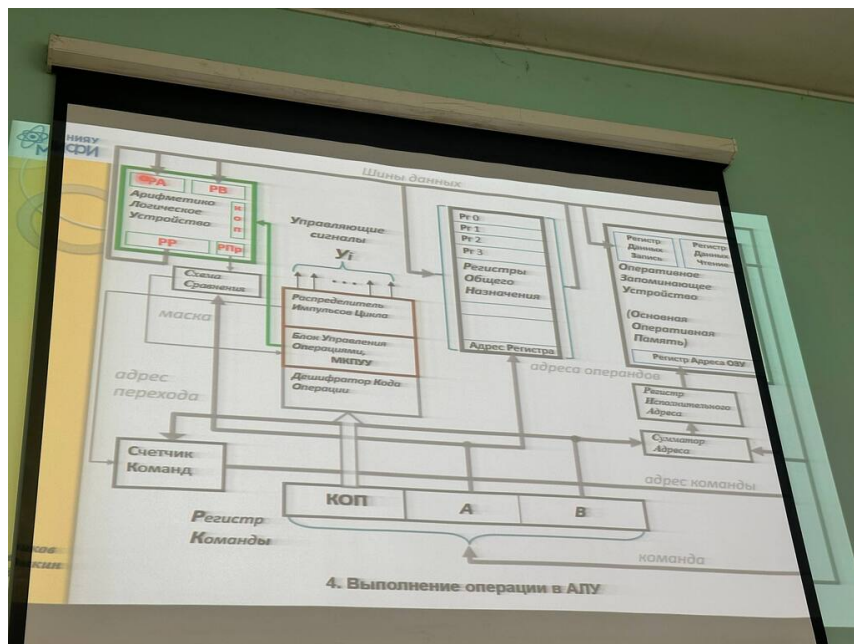




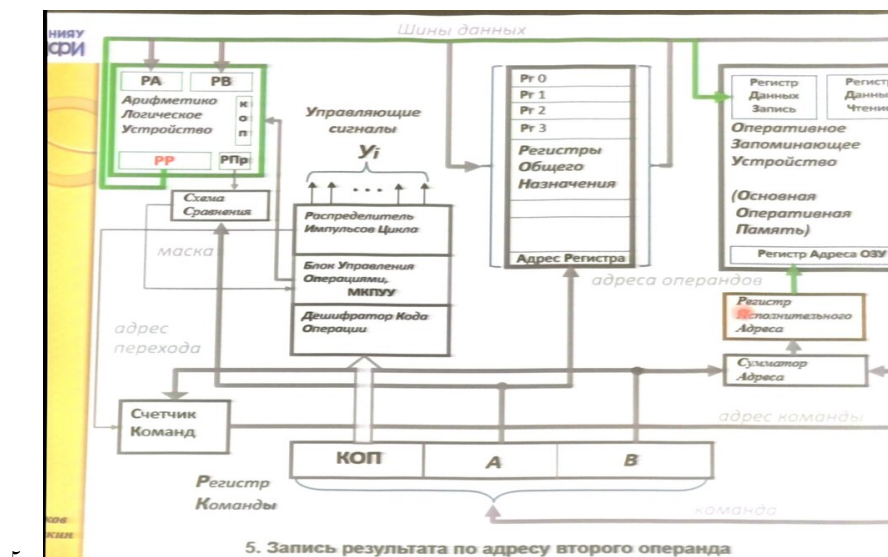
2.



3.



4.



5.



6.

Когда 2 лучше, чем 3? Что такое хорошо, а что такое плохо, когда речь идёт об ЭВМ? Что мы убрали и почему оно работает? Что со временем в этом случае? Как организовать регистр адреса так, чтобы в него входило 4 стрелочки?

9 Схемотехническая база ЭВМ

Схемотехника — наука о том, как создавать электронные устройства путём объединения компонентов в изделия с помощью схем. **Схема** — будем понимать как совокупность элементов и связей между ними.

Схемы создаются двумя способами:

1. Берётся то, на чём писать, и то, чем писать, и создаётся схема. При этом компоненты, являющиеся частями этой схемы, изображаются с помощью условных графических обозначений, а связи между ними — линиями, соединяющими входы и выходы.
2. Описание схем на специальных языках. У всех таких языков одно и то же предназначение: HDL (Hardware Description Language). Мы используем VHDL. Эти языки чем-то напоминают языки программирования, но разработка устройств не превратилась в программирование. Это лишь способ воплотить идеи разработчика.

Вся схемотехника состоит из двух частей:

1. Комбинационная схема
2. Триггерная ячейка (триггер)

С комбинационной схемой мы уже знакомы. Это не что иное, как материализация в виде схемы функций алгебры логики. Со второй частью (триггером) дело обстоит иначе: он изготавливается из того же материала, что и комбинационные схемы, а потому следует пробежаться по всему, что мы изучили.

9.1 Комбинационная схема

Комбинационная схема — это такая цифровая электронная схема, состояние выходов которой (сигнал) зависит **только** от состояния её входов (сигналов).

Следствия:

1. В такой схеме отсутствуют обратные связи (нет соединения выходов со входами).
2. В ней полностью отсутствуют элементы памяти.

Комбинационная схема — это реализация в виде схемы функции алгебры логики. Следовательно, она возникает в результате решения задачи синтеза логической схемы.

Синтез схемы состоит из следующих этапов:

1. Запись логической функции, описывающей желаемое поведение схемы, в каноническом виде (с помощью таблицы истинности). Возможны два варианта: полностью и неполностью определённые логические функции. Преобладают неполностью определённые логические функции.
2. Минимизация (оптимизация) логической функции. **Оптимизация** — получение наилучших характеристик устройства при заданных ограничениях.
3. Построение схемы цифрового устройства (логической схемы).
4. Верификация (проверка). Выполняется с помощью различных методов моделирования. **Модель** — аналог объекта в природе, реализованный в других физических явлениях. Наибольшее количество ошибок выявляется на этапе функционального моделирования. **Моделирование** — создание модели для проверки логических функций. Все элементы считаются идеальными, длины связей не учитываются, поскольку предполагается, что скорость распространения сигнала бесконечно велика. После этого обычно происходит привязка к элементной базе либо отладка на макете. Когда всё готово, изготавливаются рабочие экземпляры.

При отладке устройства решается задача анализа: определить, какую функцию оно реализует.

Комбинационные схемы строятся из логических элементов. **Логический элемент** — устройство, реализующее операции над сигналами, несущими значения логических переменных. Значения логических переменных представляются в виде электрических потенциалов. **Импульс** — изменение электрического сигнала, несущего значение логической переменной, сначала на противоположное, а затем обратно. Импульс без времени невозможен.

Модель элемента состоит из его логической функции и задержки. Логические элементы реализуются с помощью транзисторов (ТТЛ или полевые транзисторы).

Основные логические элементы:

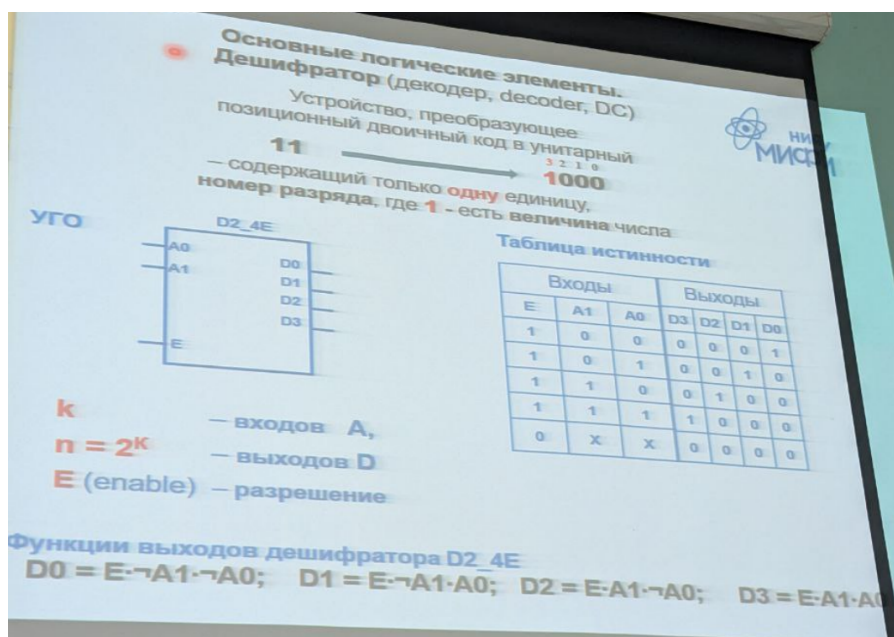
- И
- ИЛИ
- НЕ
- Штрих Шеффера
- Стрелка Пирса

Они должны составлять функционально полный базис.

Однако некоторые схемотехнические решения встречаются настолько часто, что выпускаются в виде готовых микросхем, библиотечных компонентов для систем моделирования или в виде IP-ядер (Intellectual Property core).

9.2 Комбинационные схемы средней степени интеграции

9.2.1 Дешифратор



Дешифратор — комбинационная схема, осуществляющая преобразование позиционного двоичного кода в унитарный код. **Позиционный код** — представление числа в позиционной системе счисления. **Унитарный код** — код, содержащий ровно одну единицу. Его величина определяется десятичным номером разряда, в котором находится эта единица. Входы обозначаются буквой A; их всего k штук. Во всех случаях, не указанных в таблице, на выходах возвращается 0.

Основное применение дешифратора — решение задачи выбора. Входы дешифратора соединяются со входами разрешения устройств.

9.2.2 Мультиплексор

Мультиплексор — комбинационная схема, реализующая цифровой коммутатор по схеме «много в один» (много входов → один выход).



Устройство: имеется набор информационных входов и набор адресных (селекторных) входов, а также один выход. Мультиплексор осуществляет цифровую коммутацию. Таким образом, он является переключателем, управляемым посредством указания номера на адресных входах.

9.2.3 Демультиплексор

Демультиплексор — цифровой коммутатор, работающий по схеме «один вход → много выходов».

Основные логические элементы.
Демultipлексор (DMUX, DMS)

Демultipлексор выполняет функцию, обратную функции мультиплексора. Имеет один вход **D** данных, **n** адресных входов **A_i** и **n=2ⁿ** выходов **F_i** данных.

Таблица истинности

Входы			Выходы			
D/E	A1	A0	F/D3	F/D2	F/D1	F/D0
0	0	0	0	0	0	0
1	0	0	0	0	0	1
0	0	1	0	0	0	0
1	0	1	0	0	1	0
0	1	0	0	0	0	0
1	1	0	0	1	0	0
0	1	1	0	0	0	0
1	1	1	1	0	0	0

УГО

Реализуется на дешифраторе путем переобозначения функций

9.2.4 Сумматор

Основные логические элементы.
Сумматор

Таблица истинности

N ₁₆	A	B	CI	S	CO
0	0	0	0	0	0
1	0	0	1	1	0
2	0	1	0	1	0
3	0	1	1	0	1
4	1	0	0	1	0
5	1	0	1	0	1
6	1	1	0	0	1
7	1	1	1	1	1

УГО

Логическая схема

Функция

$$S = \overline{A}B\overline{C}I + \overline{A}B\overline{C}I + \overline{A}B\overline{C}I + \overline{A}B\overline{C}I$$

$$CO = \overline{A}B\overline{C}I + \overline{A}B\overline{C}I + \overline{A}B\overline{C}I + \overline{A}B\overline{C}I$$

9.2.5 Мультиплексная схема передачи данных

Основные логические элементы
ОРГАНИЗАЦИЯ МУЛЬТИПЛЕКСНОЙ ШИНЫ ПЕРЕДАЧИ ДАННЫХ

Шина передачи данных
на мультиплексоре и демultipлексоре

На входе — мультиплексор, на выходе — демultipлексор; между ними — одна единственная линия связи.

Необходимо синхронизировать значения на адресных (селекторных) входах мультиплексора и демultipлексора.

Технология — **мультиплексирование данных** — передача через ограниченный канал за счёт разделения во времени. Осуществляется параллельно-последовательное и последовательно-параллельное преобразование кодов.

9.3 Триггерные (последовательностные) схемы

Триггерные схемы — последовательностные схемы (схемы с памятью). **Свойство памяти** — способность сохранять своё состояние во времени.

Мы будем обозначать состояние как Q ; оно может принимать два значения: логическая 1 и логический 0. Время в ЭВМ дискретно (квантовано), поэтому, когда мы говорим о времени, к состоянию следует добавлять зависимость от времени: $Q(t)$. Также используется обозначение $Q(t+1)$ — значение в следующий дискретный момент времени.

Если $Q(t) = Q(t+1)$, это означает, что состояние не изменилось, то есть оно сохранилось. Это позволяет утверждать, что память присутствует: состояние запомнилось. — Новиков Г. Г.

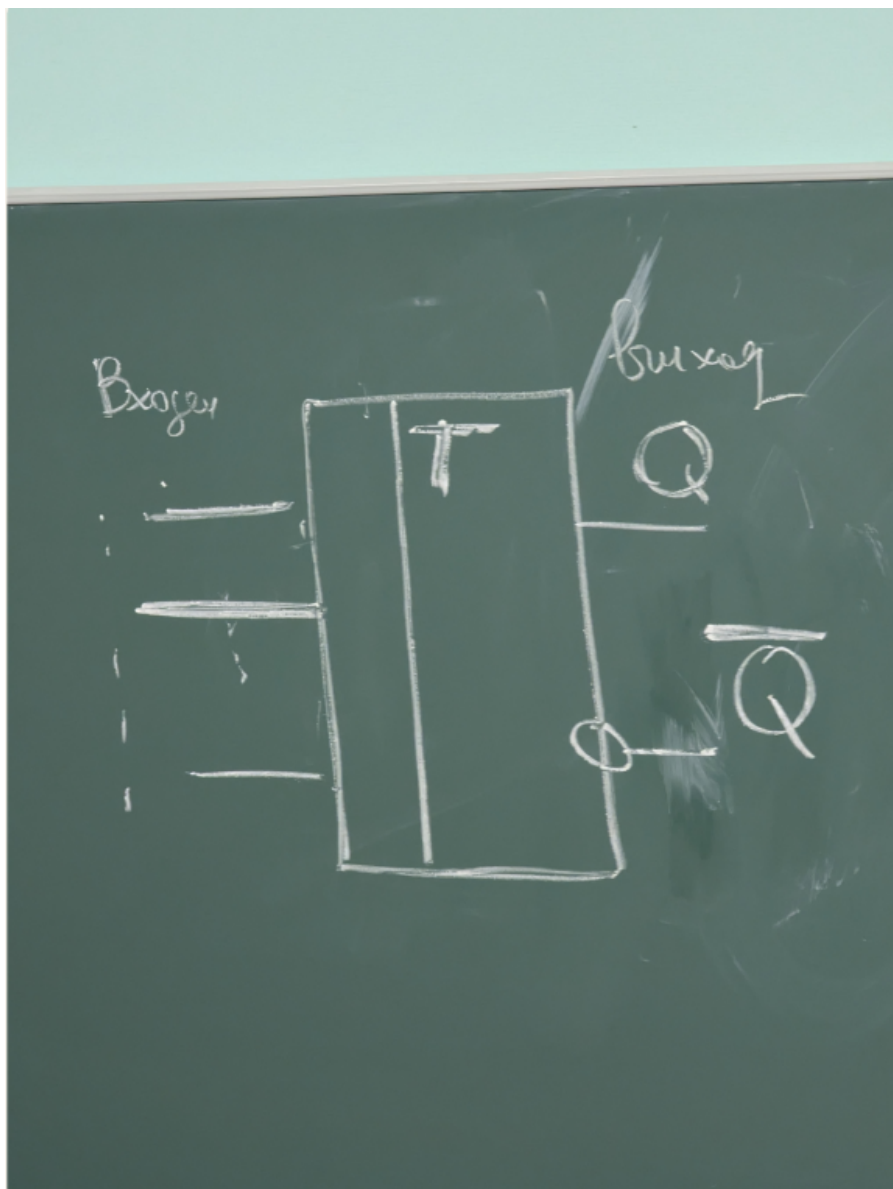
В зависимости от того, как трактуется момент перехода от текущего состояния к следующему, схемы с памятью делятся на два класса:

1. Синхронные
2. Асинхронные

Если представить схему в виде графического обозначения, то у неё всегда будет некоторый набор входов и хотя бы один выход. Состояние устройства обычно обозначается значением логической переменной на его выходе.

Если в схеме присутствует хотя бы один специальный вход (обозначаемый C) — **схема синхронная**. Если такого специального входа нет — **схема асинхронная**.

Что такое триггер (переключатель)? Это устройство, имеющее два устойчивых состояния (бистабильность), два взаимно инверсных выхода и управляющие входы (обозначаются T).



Триггерные схемы

Элементарные триггерные ячейки
на элементах ИЛИ-НЕ

ИЛИ-НЕ

A	B	A + B
0	0	1
0	1	0
1	0	0
1	1	0

Полная таблица переходов

N	S	R	Q_t	Q_{t-1}	
0	0	0	0	0	Q_t
1	0	0	1	1	
2	0	1	0	0	0
3	0	1	1	0	
4	1	0	0	1	1
5	1	0	1	1	
6	1	1	0	X	X
7	1	1	1	X	

X — запрещенное состояние входов

УГО

Таблица переходов

t		t+1
S	R	Q(t)
0	0	Q(t)
0	1	0
1	0	1
1	1	X

RS — триггер с прямыми входами на элементах ИЛИ-НЕ, УГО, таблица переходов

Вопросики 04.12.2025

Чем отличается T от $T + 1$ - T - сейчас, $T + 1$ - следующий момент времени. Что может вызвать переход времени из состояния T в состояние $T + 1$ - 2 варианта, которые определяют 2 вида схем: синхронные и

асинфронные.

Для асинхронных: любое изменение сигналов на любом входе. Для синхронных: всё то же самое, но на определённом конкретном коде (синхронизация). Синхронно - одновременно.

Что такое последовательные схемы? - сейчас состояние было, сейчас пришло воздействие, а изменение будет только в следующий момент времени. Простейший представитель последовательных схем? - триггер.

9.3.1 Обобщённая структура триггрной схемы

такое преобразование - комбинационная схема, которая превращает всё в 2 функции, которые работают с R, S входами. Те кто

9.3.2 Элементарные триггерные схемы. Основные типы.

1. Т - триггер. Т от слова toggle (перевёртыш). Если проанализировать таблицу переходов, то мы понимаем, что когда приходит 0, то он в состоянии хранения, а при 1 формируется НЕ Q(t). Кроме того есть вход С - синхронизация. Как правило для корректного выполнения операции смысловые входы должны быть установлены перед синхронизацией. То минимальное время за которое это должно быть установлены сигналы на управляющий входах - время предустановки.
2. RS(SR) - триггер. Рассмотрен выше.
3. JK - триггер. J и K ведут себя так же как и S R. Отличие в запрещённой комбинации. Вместо неё в JK тигер типа Т. Запрещённая комбинация приводит к изменению состояния на противоположное.
4. D - триггер. Delay (задержка) В момент T+1 будет на выходе то, что было на входе в момент времени T. То есть хранение.
5. DV - триггер. D - триггер с входом разрешения. До тех пор пока на входе V присутствует 0 триггер находится в состоянии хранения. Когда 1, он повторяет D триггер.

9.4 Регистры

Берём триггеры и делаем из них регистры. Кроме того регистры могут

1. Хранение (количество триггеров = количеству разрядов)
2. Сдвиг (у регистров сдвига)
3. Универсальные регистры - регистры способные выполнять микрооперации

Примеры регистров:

- Регистр хранения

CLS - вход асинхронной команды очистки в 0.
Состоит из 4 D триггеров.

- Регистр сдвига

SLI - Shift Left Input - вход последовательного приёма данных при сдвиге влево. Название означает, что происходит сдвиг влево (в сторону старших разрядов/сверху вниз по картинке).
D0, D1, D2, D3 - входы для параллельной загрузки (тоже самое у регистра хранения).
L - 1 разрешает load (загрузку) если 0, то выполняется функция сдвиг.
CE - chip enable - разрешающий работу вход. С - вход с треугольничков внутрь значит, что все мероприятия происходят по команде синхроимпульса.
Состоит из 4 Т-триггеров и 4 мультиплексоров, объединённых селекторным входом (из L).

9.5 Счётчик

Устройство последовательностного типа, обладающее способностью считать. Предназначено для подсчётов импульсов, приходящий на специальный вход (счётный вход) устройства.

Помимо подсчёта это устройство

- Хранит то, что насчитала
- Отображается на выходах в заданном коде (кодировке).
- Имеет конечное количество состояний, то есть чисел, которые в нём могут быть, определяемое количество разряда счётчика. Достигнув максимального значения счётчик сбрасывается. Количество неповторяющихся состояний - модуль счётчика (модель пересчёта). Из графа однозначно следует, что следующее состояние счётчика зависит только от предыдущего состояния и не зависит ни от чего пришедшего из вне.

10 Лекция от 18.12.2025

Что такое магистраль? Что такое сегментация? Какая проблема привела к тому, что пришлось сделать память сгигентированной? Одна из проблем: нужно из нано мира передать информацию в микро мир. Что за приём позволяет через узкое горлышко протаскать что-то крупноразмерное? - мультиплексирование. Как сделать команду в Itnel 8086, которая переносила бы 1 байн из одной ячейки памяти в другую? - сдвиг. Правильный ответ никак (или 2 команды из ячейки в регистр и из регистра в ячейку) Какие бывают форматы команд двухадресных в этом устройстве: РР - один операнд в 1 регистре второй в другом регистре (регистр регистр)

РС - регистр storage. Один в регистре второй в памяти. СС - обы в оперативной памяти. **Этого формата нет** в процессоре Intel 8086. Обязательно один из операндов находится **в регистре**.

Наш ребёнок получился мертворождённым. Нужно добавить software чтобы устройство стало полезным.

10.1 Software. Програмное обеспечение.

Без аппаратного обеспечения тоже абстрактна и бесполезна. Без программного обеспечения железка - дорогой камень. Вместе очень классная вещь. **Всё что можно реализовать программно можно реализовать аппаратно и наоборот.**

Предварительные условия:

1. Программа и данные уже находятся в оперативной памяти.
2. Адрес первой команды программы уже находится в счётчике. (магия)

Теперь разберёмся как оно туда попало. Счётчик состоит из столькох триггеров какой разрядности необходим счётчик. Для адреса команды необходимо логорифм двоичный от области памяти. Без питания с триггерами ничего не происходит. А при включении питания определяется абсолютно случайно.

Получается, что нам необходимо загрузить программу и каким-то образом указать счётчику команд с какой команды выполнять программу.

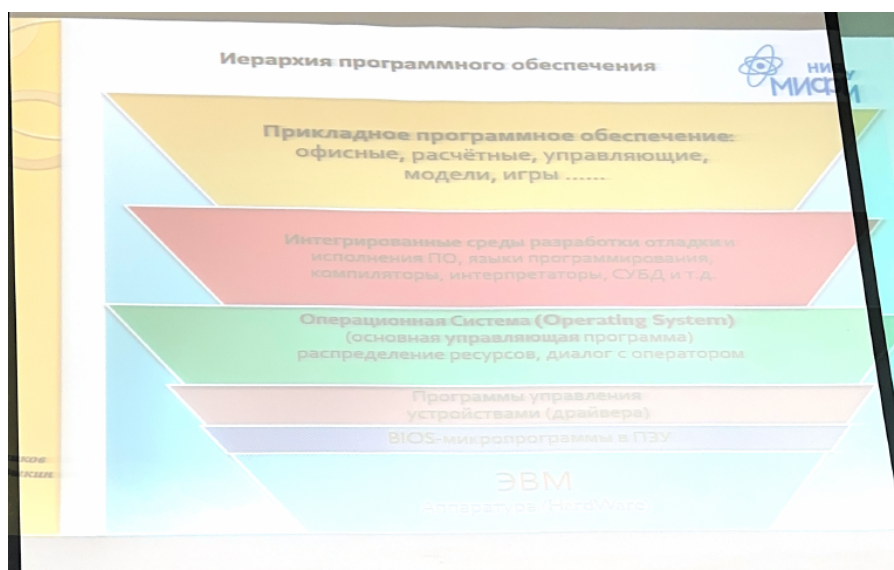
Делается это следующим образом: В ООП (RAM): по Нейману рядом находится **энергонезависимая** память. Осталось как-то доставить в ООП. Ключевая роль у источника питания. Как только он приходит в рабочий режим и достигает нужных напряжений он выдаёт на специальном выходе импульс, который называется общесистемный сброс. Этот импульс передаётся в те места, где необходимо установить всё в жedaемое исходное состояние. Так же можно вручную пустить этот сигнал кнопкой **resrt**. Главное что делает этот импульс: добирается до счётчика команд и как правило попадает на его вход R (CLR). После чего счётчик оказывается в состоянии все 0. Дальше нужно поработать с памятью. Отрезать кусочек и сдлеать его не RAM, а ROM (Read only memory). Как правило абсолютно энергонезависимая. Именно туда с ячейки 0 укладывается та программа, которая должна запуститься первой. А дальше то, что

написано в тексте этой программы. (Слазить в энергонезависимую память и загрузить в ООП всё что надо). Base Input Output System (BIOS). Таким образом решается проблема откуда в памяти появляется программа (данные) и заполняется счётчик.

10.2 Организация вычислительного процесса

1. Иерархия программного обеспечения
2. Понятие операционная система
3. Монопольный и Мультипрограммный режим работы ЭВМ.
4. Распределение ресурсов
5. Виртуализация и защита памяти
6. Конвейнерная организация ЭВМ

10.2.1 Иерархия программного обеспечения.



Драйвер - программа, которая управляет тем конкретным устройством, для которого она сделана и реализует 2 интерфейса: с одной стороны устройство, с другой следующий слой программного обеспечения. Суть: разную специфику разных периферийных устройств привести к стандарту. Основная управляющая программа - операционная система. Смысл: обеспечение и максимальный комфорт жизнедеятельности других программ. Прикладное программное обеспечение: то что даёт какую-то пользу. Где-то между ними живут ещё большое количество программ, которые можно называть инструментальными программами (главным образом для написания, отладки и разработки других программ. БД, ЯП, системы автоматизации проектирования, ...)

Специфика организации этого определяет как стиль общения так и технические характеристики вычислительной системы.

Большую часть составляют интерфейсы системы общения пользователя и системы. (перфокарты, голос, камеры и т.д.)

Вся эта деятельность ОС сводится к несложным вещам:

1. Процесс
2. Ресурс

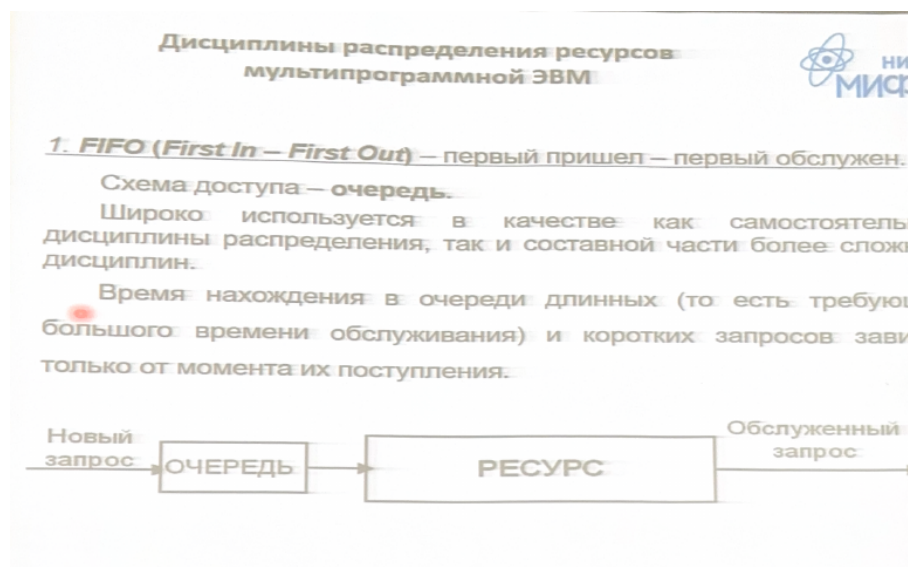
10.2.2 ОС

10.3 Распределение ресурсов между процессами.

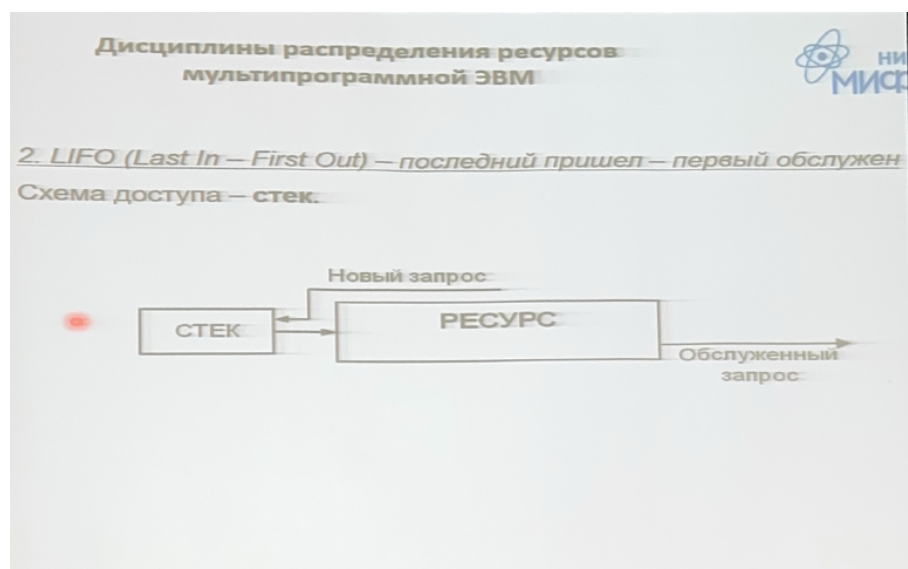
Есть некий процесс, который ждёт ответ от пользователя (захватил экран и клавиатуру). То есть другие процессы не могут получить к ним доступ. Решение: Например выдавать доступ на лимитированный промежуток времени.

10.3.1 Процедуры распределения ресурсов мультимпрограмной ЭВМ.

Например FIFO (очередь)



В столовой тянемся к ресурсом, но кто первый встал тому и тапки. Но есть процессы у которых приоритет выше других. Его пропускают вперед. Но если авторитетных много, то через одного. LIFO. Первый вылетает последний патрон. Stack.



10.4 Дисциплины с многими очередями.



Основа дисциплины — **круговой циклический алгоритм**.

Все новые запросы поступают в очередь 1.

Время, выделяемое на обслуживание любого запроса, равно длительности кванта t_k . Если запрос обслужен за это время, то покидает систему, а если нет, то по истечении выделенного кванта времени он поступает в конец очереди $i+1$.

На обслуживание выбирается запрос из очереди i , только если очереди 1, ..., $i-1$ пусты.

Таким образом, длинные запросы поступают сначала в очередь 1, затем постепенно доходят до очереди N и здесь обслуживаются до конца либо по дисциплине *FIFO*, либо по круговому циклическому алгоритму.

Данная дисциплина может работать как с относительными, так и с абсолютными приоритетами программ.

1.1. Обслуживание процессов с **относительными приоритетами**. Заявка, входящая в систему, не вызывает прерывания обслуживаемой заявки, даже если последняя и менее приоритетна. Только после окончания обслуживания текущей заявки начинается обслуживание более приоритетной.

1.2. Обслуживание процессов с **абсолютными приоритетами**. Если во время обслуживания программы из очереди i в очередь с большим приоритетом поступает новая программа, то после окончания текущего кванта t_k обрабатываемая программа прерывается и возвращается в начало своей очереди, с тем чтобы впоследствии дообслужиться на время, недобранное до $2^{i-1} \cdot t_k$.

10.5 Виртуализация и защита памяти.

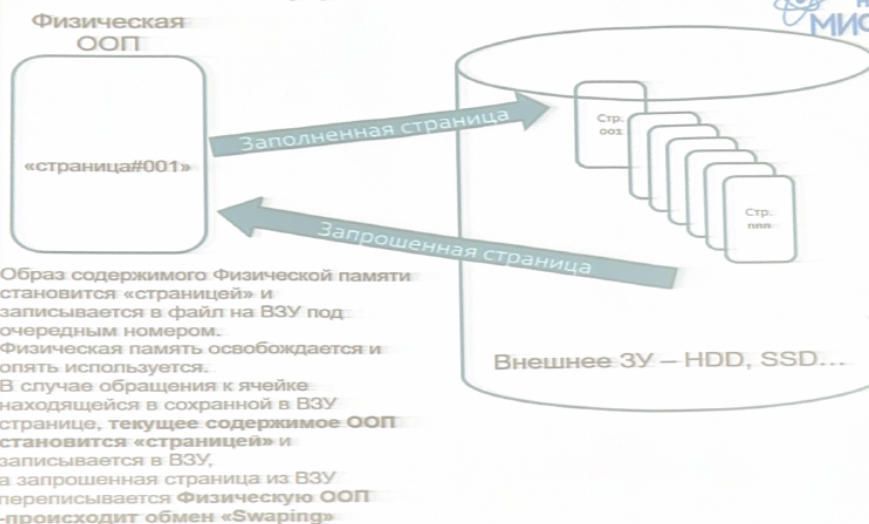
Попытка решения 1 из основных вопросов философии. Память мы хотим быструю и бесконечную, а технологии не позволяют :-(((

Виртуализация и защита памяти



Виртуализацией называется механизм, в составе операционной системы, обеспечивающий расширение оперативной памяти за счет памяти на внешнем носителе (обычно HDD). Путем организации динамического обмена образами ООП (называемыми страницами) с файлом, расположенным на диске (страничный файл, файл подкачки), у пользователя создается иллюзия доступа к памяти существенно превышающей размер физической ООП ЭВМ к – «Виртуальной памяти».

Виртуальная память



Защита памяти



Защита памяти – механизм в составе операционной системы обеспечивающий ограничение доступа к отдельным областям памяти ЭВМ. Основное назначение – это сохранение целостности данных, от случайного или преднамеренного изменения при мультипрограммном режиме работы ЭВМ.

Два основных вида защиты:

- Защита при управлении памятью (границные регистры)
- Защита по привилегиям (кольца защиты)

Метод граничных регистров заключается во введении двух граничных регистров, указывающих верхнюю и нижнюю границы области памяти, куда программа имеет право доступа.



Защита памяти по привилегиям

